BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 468 (July 2025)

VLSI Laboratory

Final Project Report

Major**: Electronics(G2)** Project Group**: 02**

Design and Implementation of UART Communication Interface

Course Instructors:

Nafis Sadik
(Assistant Professor)

Archisman Sarkar
(Part-time Lecturer)

Signature of Instructor: _____

Submitted By:

Md. Shahriar Rahman Siam	2006108	Sheikh Munkasir Ahmed Rafeed	2006109
Nafisa Anjum Promi	2006163	Mst.Tasnim Mehedy	2006171

Date of Submission: 01 April 2026

Academic Honesty Statement:

"In signing this statement, We, hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: 

Full Name: Md. Shahriar Rahman Siam

Student ID: 2006108

Signature: 

Full Name: Nafisa Anjum Promi

Student ID: 2006163

Signature: 

Full Name: Sheikh Munkasir Ahmed Rafeed

Student ID: 2006109

Signature: 

Full Name: Mst. Tasnim Mehedy

Student ID: 2006171

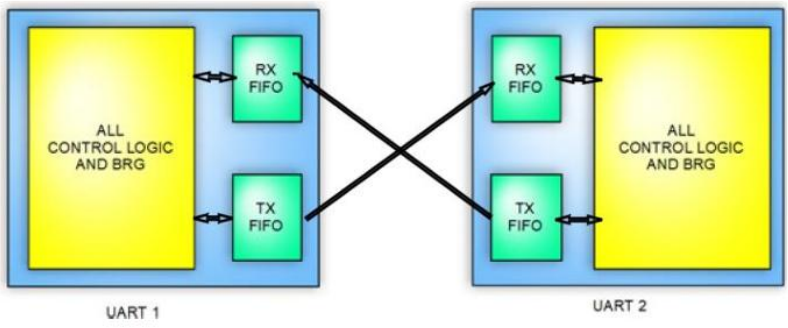
Table of Contents

1 Abstract

Universal Asynchronous Receiver Transmitter (UART) is a widely used serial communication protocol for reliable data exchange between digital systems. This project presents the design, comprehensive verification, and logic synthesis of an 8-bit Universal Asynchronous Receiver-Transmitter (UART) module. The core functionality was initially validated through a directed testbench executing six targeted scenarios, successfully verifying operations such as basic data loopback, mid-transmission resets, back-to-back transfers, and 16x receiver oversampling. To ensure robustness, a layered verification environment was then developed to apply Constrained Random Verification (CRV). By running 500 randomized transactions targeting specific data corner cases, including all zeros, all ones, and alternating bit patterns, the testing model successfully achieved 100% functional coverage. Finally, the design underwent rigorous gate-level synthesis using the Genus tool to evaluate power and area trade-offs across varying effort levels, clock periods, and timing delays. The optimal synthesis configuration utilized a high effort level alongside a 100 ns clock period (10 MHz frequency). This specific balance of parameters resulted in a highly power-efficient layout boasting a total power consumption of 7478.243 nW and a compact cell area of 826 μm^2.

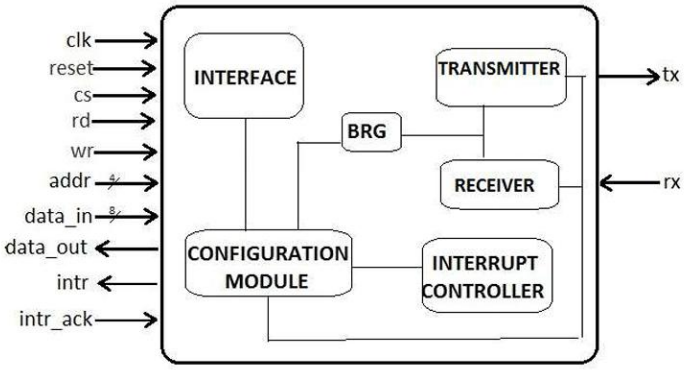
2 Introduction

Serial communication plays a fundamental role in modern digital systems, enabling efficient data exchange between microcontrollers, embedded devices, and peripheral components. Among various serial communication protocols, the Universal Asynchronous Receiver Transmitter (UART) is one of the most widely used due to its simplicity, low hardware requirements, and ease of implementation. Unlike synchronous protocols, UART does not require a shared clock signal; instead, it relies on predefined baud rates for timing synchronization between transmitting and receiving devices. This makes UART highly suitable for short-distance communication in embedded and integrated systems.



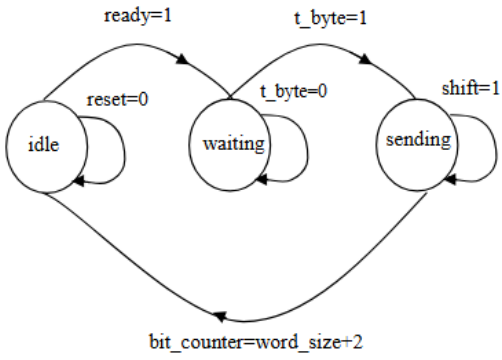
Generic double UART Block Diagram

In this project, a complete UART communication interface is designed and implemented using the Verilog Hardware Description Language (HDL). The system consists of three main modules: a transmitter (TX), a receiver (RX), and a baud rate generator. The transmitter converts 8-bit parallel input data into a serial stream by appending start and stop bits, while the receiver reconstructs the original data by oversampling the incoming serial signal at 16 times the baud rate. To effectively isolate and validate the core logic, the design utilizes an internal loopback architecture where the transmitter and receiver are instantiated within a single top-level module. This creates a modular, synthesizable architecture focused on accurate timing, reliable data transfer, and efficient hardware utilization.

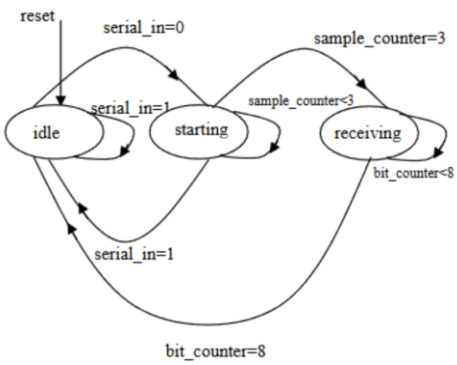


.UART Module

To validate the functionality of the proposed design, both directed and layered verification methodologies are employed. The directed testbench is used to verify fundamental operations using predefined input sequences. Building upon this, a layered testbench environment applies Constrained Random Verification (CRV), successfully achieving 100% functional coverage across 500 randomized transactions targeting specific data corner cases. Finally, the design is synthesized to evaluate and optimize key performance parameters. By analyzing area, timing, and power trade-offs across different effort levels and clock periods, the project demonstrates a complete digital design flow.



State diagram of UART transmitter



State diagram of UART receiver

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The scope of this project is to design, verify, and synthesize a Universal Asynchronous Receiver Transmitter (UART) communication interface using Verilog Hardware Description Language (HDL). The primary focus is on developing a reliable and modular digital system capable of performing asynchronous serial data transmission and reception between two devices. The design includes the implementation of key functional blocks such as the UART Transmitter (TX), UART Receiver (RX), and a Baud Rate Generator to ensure proper timing and synchronization of data transfer.

This project emphasizes functional correctness and hardware efficiency by incorporating structured verification methodologies. Both directed and layered testbenches are utilized to validate the design under different operating conditions, ensuring accurate data transmission, proper detection of start and stop bits, and correct reconstruction of received data. Waveform analysis is performed to observe signal behavior and confirm compliance with UART communication standards.

In addition to functional verification, the scope also includes synthesis and optimization of the design to evaluate hardware performance metrics such as area, timing, and power consumption. The synthesized design is analyzed to ensure that it meets design constraints and operates reliably within specified limits. While the current work focuses on RTL design, verification, and synthesis, it also considers the extension toward physical design implementation, making the project relevant for complete VLSI design flow and practical digital system applications.

3.1.2 Literature Review

Previous research and technical literature have extensively explored the design and implementation of Universal Asynchronous Receiver Transmitter (UART) systems, particularly in the context of digital communication and FPGA-based designs. These studies provide a strong foundation in asynchronous serial communication, including concepts such as baud rate generation, start and stop bit detection, and data synchronization. They also highlight important design considerations such as modular architecture, timing accuracy, and efficient hardware utilization. Furthermore, research emphasizes the importance of verification methodologies and synthesis optimization to ensure reliable operation and improved performance in terms of area, timing, and power. Simulation-based validation and hardware-oriented implementations demonstrate how theoretical concepts are translated into

practical digital systems. This body of literature serves as a valuable guide for developing efficient, reliable, and scalable UART communication interfaces. Here are some of the studies conducted in this respect:-

[1] Digital System Design of FPGA-Based UART Protocol Using Verilog HDL
This paper presents the design and implementation of a UART protocol using Verilog HDL on an FPGA platform. It focuses on modular design techniques, including separate transmitter, receiver, and baud rate generator modules. The study demonstrates reliable serial communication through simulation and highlights the importance of structured RTL design for efficient hardware realization.

[2] Universal Asynchronous Receiver Transmitter (UART)
This work provides a detailed overview of UART communication principles, including frame structure, timing mechanisms, and asynchronous data transfer. It explains the roles of start and stop bits, baud rate synchronization, and basic transmitter-receiver operation. The paper serves as a strong theoretical reference for understanding UART fundamentals.

[3] Design and Implementation of UART Using Verilog
This research focuses on the practical implementation of a UART system using Verilog HDL. It emphasizes functional correctness, modular architecture, and simulation-based verification. The design is validated through waveform analysis, demonstrating accurate serial data transmission and reception.

[4] UART IP Core Design and Verification Using SystemVerilog
This study presents an advanced approach to UART verification using SystemVerilog methodologies. It highlights the importance of structured and reusable testbenches, including layered verification techniques. The work aligns with modern verification practices and ensures robust validation of UART functionality.

[5] Verification of UART Using SystemVerilog
This paper focuses on verification strategies for UART designs, emphasizing the use of systematic test environments to validate different operating conditions. It demonstrates how structured verification improves reliability and helps detect functional errors effectively.

[6] High-Speed Low-Power UART Design Using Verilog
This work explores optimization techniques in UART design, focusing on improving performance metrics such as speed, area, and power consumption. It highlights synthesis-level improvements and demonstrates how efficient design practices can lead to optimized hardware implementation.

[7] Design of Anti-Interference UART Receiver
This study presents enhancements to the UART receiver to improve robustness against noise and interference. It focuses on sampling techniques and error-resistant design strategies, making the UART system more reliable in practical communication environments.

[8] Design and Implementation of UART Serial Communication Module Based on FPGA
This paper demonstrates a complete UART implementation on FPGA, including design, simulation, and testing. It highlights real-world applicability and shows how UART modules can be integrated into larger digital systems.

3.1.3 Formulation of Problem

In modern digital and embedded systems, efficient and reliable communication between devices is essential, yet many communication protocols require complex hardware, strict clock synchronization, or higher power consumption, making them less suitable for simple and resource-constrained applications. Although the Universal Asynchronous Receiver Transmitter (UART) provides a simpler solution through asynchronous communication, designing a robust UART interface that ensures accurate data transmission and reception while maintaining proper timing and hardware efficiency remains a key challenge. The problem addressed in this project is to develop a fully functional, modular, and synthesizable UART communication interface that can correctly handle start and stop bit detection, generate appropriate baud rates for synchronization, and accurately reconstruct received data without errors. Additionally, the design must be thoroughly verified using both directed and layered testbenches to ensure functional correctness under various conditions, and it must be synthesized and optimized to evaluate and improve performance metrics such as area, timing, and power consumption. The problem further extends to preparing the design for potential physical implementation, thereby addressing the complete VLSI design flow and ensuring that the UART system is reliable, efficient, and suitable for integration into larger digital systems.

3.2 Design Method

The design of the UART communication interface is carried out using a structured and modular approach in Verilog HDL, focusing on clarity, functionality, and synthesizability. The overall system is divided into three main components: the UART Transmitter (TX), UART Receiver (RX), and Baud Rate Generator, each implemented as independent modules to ensure flexibility and ease of verification. The transmitter is designed to convert parallel input data into a serial bit stream by appending start and stop bits, while the receiver detects the start bit, samples incoming data based on the generated baud rate, and reconstructs the original parallel data. The baud rate generator is implemented using a counter-based design to produce accurate timing signals required for both transmission and reception. The control logic of the system is implemented using sequential and state-based operations to ensure proper data flow and synchronization between modules. Functional verification is performed using both directed and layered testbenches, where the directed approach validates basic functionality and the layered approach provides a structured environment for comprehensive testing. Simulation results are analyzed through waveform observation to confirm correct operation. Finally, the design is synthesized to evaluate performance in terms of area,

timing, and power, and optimization techniques are applied to improve hardware efficiency, ensuring that the design is suitable for implementation in real digital systems.

3.3 Simulation Model

The simulation model of the UART communication interface is developed to validate the functional correctness of the design prior to hardware implementation. The complete system, including the transmitter, receiver, and baud rate generator modules, is described using Verilog HDL and simulated using a digital simulation tool. A clock signal is generated to drive the synchronous logic, and an active reset is applied to initialize the system into a known state. Input stimulus is provided through a directed testbench, where predefined data patterns are transmitted to verify correct serial data generation and reception. In addition, a layered testbench environment is used to apply structured and scalable test scenarios, ensuring comprehensive verification under different operating conditions. The serial output of the transmitter is internally connected to the receiver input to form a loopback configuration, enabling end-to-end validation of the communication process. Simulation results are analyzed through waveform observation, where key signals such as start bit detection, data bits, stop bit, busy signal, and ready signal are monitored to confirm proper UART operation. Following successful simulation, the design is synthesized and prepared for physical design considerations, highlighting its readiness for further stages such as placement, routing, and implementation in real hardware.

4 Implementation

4.1 DIRECTED TESTBENCH

A directed testbench is used to verify the functional correctness of the UART design by applying predefined input stimuli and observing the corresponding outputs. In this approach, specific test cases are manually created to check key operations such as data transmission, reception, start and stop bit handling, and system response under reset conditions. The testbench drives input signals like data input, write enable, and reset, while monitoring output signals such as transmitted data, received data, busy, and ready flags. By comparing the expected and actual outputs through waveform analysis, the directed testbench ensures that the UART modules operate correctly under controlled scenarios before moving to more advanced verification techniques.

Verilog Code for UART

```
`timescale 1ns / 1ps

//
```

```
//=============================================================================
// SECTION 1: TOP LEVEL MODULE
//=============================================================================
module uart_top (
    input rst,
    input [7:0] data_in,
    input wr_en,
    input clk,
    input rdy_clr,
    output rdy,
    output busy,
    output [7:0] data_out
);

    wire rx_clk_en;
    wire tx_clk_en;
    wire tx_temp;

    // Instantiate Sub-modules
    baud_rate_genrator bg (
        .clock(clk),
        .reset(rst),
        .enb_tx(tx_clk_en),
        .enb_rx(rx_clk_en)
    );

    uart_sender us (
        .clk(clk),
        .wr_en(wr_en),
        .enb(tx_clk_en),
        .rst(rst),
        .data_in(data_in),
        .tx(tx_temp),
        .tx_busy(busy)
    );

    uart_reciever ur (
        .clk(clk),
        .rst(rst),
        .rx(tx_temp),
        .rdy_clr(rdy_clr),
        .clken(rx_clk_en),
        .rdy(rdy),
        .data_out(data_out)
    );

endmodule
```

```
//
================================================================
=
// SECTION 2: BAUD RATE GENERATOR
//
================================================================
=
module baud_rate_genrator(
    input clock,
    input reset,
    output reg enb_tx,
    output reg enb_rx
);
    parameter clk_freq = 100000000;
    parameter baud_rate = 9600;

    reg [15:0] counter_tx;
    reg [15:0] counter_rx;

    parameter divisor_tx = clk_freq/baud_rate;
    parameter divisor_rx = clk_freq/(16 * baud_rate);

    // Transmitter Enable Logic
    always @(posedge clock) begin
        if(reset) begin
            counter_tx <= 0;
            enb_tx <= 0;
        end else if(counter_tx == divisor_tx - 1) begin
            enb_tx <= 1;
            counter_tx <= 0;
        end else begin
            counter_tx <= counter_tx + 1'b1;
            enb_tx <= 0;
        end
    end

    // Receiver Enable Logic (16x Oversampling)
    always @(posedge clock) begin
        if(reset) begin
            counter_rx <= 0;
            enb_rx <= 0;
        end else if(counter_rx == divisor_rx - 1) begin
            counter_rx <= 0;
            enb_rx <= 1;
        end else begin
            counter_rx <= counter_rx + 1;
            enb_rx <= 0;
        end
```

```
    end
endmodule


//
================================================================================
=
// SECTION 3: UART SENDER (TRANSMITTER)
//
================================================================================
=
module uart_sender(
    input clk,
    input wr_en,
    input enb,
    input rst,
    input [7:0] data_in,
    output reg tx,
    output tx_busy
);

    parameter STATE_IDLE  = 2'b00;
    parameter STATE_START = 2'b01;
    parameter STATE_DATA  = 2'b10;
    parameter STATE_STOP  = 2'b11;

    reg [7:0] data = 8'h00;
    reg [2:0] bitpos = 3'h0;
    reg [1:0] state = STATE_IDLE;

    always @(posedge clk) begin
        if(rst) begin
            tx <= 1'b1;
            state <= STATE_IDLE;
        end else begin
            case (state)
                STATE_IDLE: begin
                    if (wr_en) begin
                        state <= STATE_START;
                        data <= data_in;
                        bitpos <= 3'h0;
                    end
                end
                STATE_START: begin
                    if (enb) begin
                        tx <= 1'b0;
                        state <= STATE_DATA;
                    end
                end
                STATE_DATA: begin
```

```
                    if (enb) begin
                        tx <= data[bitpos];
                        if (bitpos == 3'h7)
                            state <= STATE_STOP;
                        else
                            bitpos <= bitpos + 3'h1;
                    end
                end
                STATE_STOP: begin
                    if (enb) begin
                        tx <= 1'b1;
                        state <= STATE_IDLE;
                    end
                end
                default: state <= STATE_IDLE;
            endcase
        end
    end

    assign tx_busy = (state != STATE_IDLE);

endmodule


//
//////////////////////////////////////////////////////////////////////
//
// SECTION 4: UART RECEIVER
//
//////////////////////////////////////////////////////////////////////
//
module uart_reciever(
    input clk,
    input rst,
    input rx,
    input rdy_clr,
    input clken,
    output reg rdy,
    output reg [7:0] data_out
);

    parameter RX_STATE_START    = 2'b00;
    parameter RX_STATE_DATA_OUT = 2'b01;
    parameter RX_STATE_STOP     = 2'b10;

    reg [1:0] state = RX_STATE_START;
    reg [3:0] sample = 0;
    reg [3:0] index = 0;
    reg [7:0] temp = 8'b0;
```

```
    always @(posedge clk) begin
        if(rst) begin
            rdy <= 0;
            data_out <= 0;
            state <= RX_STATE_START;
            sample <= 0;
        end else begin
            if (rdy_clr)
                rdy <= 0;

            if (clken) begin
                case (state)
                    RX_STATE_START: begin
                        if (!rx || sample != 0)
                            sample <= sample + 4'b1;

                        if (sample == 15) begin
                            state <= RX_STATE_DATA_OUT;
                            index <= 0;
                            sample <= 0;
                            temp <= 0;
                        end
                    end
                    RX_STATE_DATA_OUT: begin
                        sample <= sample + 4'b1;
                        if (sample == 4'h8) begin
                            temp[index] <= rx;
                            index <= index + 4'b1;
                        end
                        if (index == 8 && sample == 15)
                            state <= RX_STATE_STOP;
                    end
                    RX_STATE_STOP: begin
                        if (sample == 15 ) begin
                            state <= RX_STATE_START;
                            data_out <= temp;
                            rdy <= 1'b1;
                            sample <= 0;
                        end else begin
                            sample <= sample + 4'b1;
                        end
                    end
                    default: state <= RX_STATE_START;
                endcase
            end
        end
    end

endmodule
```

The workflow of this UART system operates as a self-contained internal loopback, orchestrated by the uart_top module that connects the transmitter's serial output directly to the receiver's serial input via the tx_temp wire. The process begins with the baud_rate_genrator dividing the 100 MHz system clock to supply a 9600 bps tick for transmission and a 16x oversampled tick for reception. When the write enable signal is triggered, the uart_sender latches the 8-bit parallel data and cycles through a Finite State Machine to serially shift out a start bit, the data bits, and a stop bit onto the tx_temp line. Concurrently, the uart_reciever detects the incoming start bit on this shared line and uses its 16x oversampled clock to accurately read each bit at its exact midpoint. Once the full 8-bit sequence and stop bit are captured, the receiver transitions to its STOP state, asserts the rdy flag to indicate success, and outputs the reconstructed parallel byte on data_out, seamlessly completing the cycle

Test Cases

Our design has been tested with 6 Test Cases:

Test Case	Purpose / Reason
Basic TX/RX Loopback	Validates the basic data path from sender to receiver.
Reset During Transmission	Ensures the state machine returns to IDLE if a reset occurs mid-frame.
Back-to-Back Transfer	Verifies that consecutive words can be sent without losing data in between.
Oversampling Verification	Checks that the receiver samples data at the midpoint of each bit.
Stop Bit Verification	Confirms the receiver only asserts rdy after the full stop bit duration.

Verilog Code for Directed Testbench

I/O Signals, DUT Instantiation & Clock Generation

```
module uart_top_tb;
    // Signals
```

```
    reg clk, rst;
    reg [7:0] data_in;
    reg wr_en;
    reg rdy_clr;
    wire rdy, busy;
    wire [7:0] dout;

    // Instantiate Top Module
    uart_top dut(
        .rst(rst),
        .data_in(data_in),
        .wr_en(wr_en),
        .clk(clk),
        .rdy_clr(rdy_clr),
        .rdy(rdy),
        .busy(busy),
        .data_out(dout)
    );

    // Clock Generation (100MHz)
    initial clk = 0;
    always #5 clk = ~clk;
```

TEST CASE 1: system reset

```
//
============================================================================
=
    // TASKS FOR DIRECTED TESTING
    //
============================================================================
=

    // System Reset Task
    task perform_reset;
        begin
            @(negedge clk);
            rst = 1'b1;
            repeat(5) @(negedge clk);
            rst = 1'b0;
            @(negedge clk);
            $display("--- System Reset Performed ---");
        end
    endtask
```

TEST CASE 2: single byte

```
// Send Single Byte Task
    task send_byte(input [7:0] din);
        begin
            wait(!busy);
            @(negedge clk);
            data_in = din;
            wr_en = 1'b1;
            @(negedge clk);
            wr_en = 1'b0;
            $display("[TX] Sending data: %h", din);
        end
    endtask
```

TEST CASE 3: clear ready flag

```
// Clear Ready Flag Task
    task clear_ready;
        begin
            wait(rdy);
            $display("[RX] Data Received: %h", dout);
            @(negedge clk);
            rdy_clr = 1'b1;
            @(negedge clk);
            rdy_clr = 1'b0;
        end
    endtask
```

Main test sequence

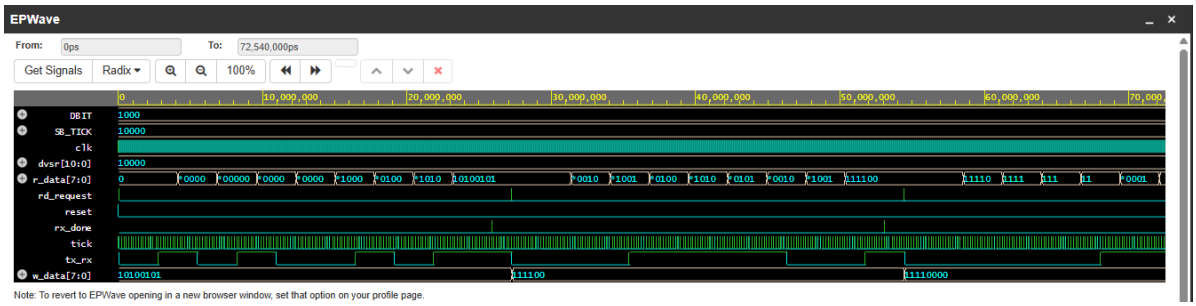
```
//
============================================================================
=
    // MAIN TEST SEQUENCE
    //
============================================================================
=
    initial begin
        $dumpfile("dump.vcd");
        $dumpvars(0, uart_top_tb);

        // Initialize inputs
        rst = 0;
        data_in = 0;
```

```
        wr_en = 0;
        rdy_clr = 0;

        // 1. Basic TX/RX Loopback (8'hA5)
        $display("\n--- Test Case 1: Basic TX/RX Loopback ---");
        perform_reset();
        send_byte(8'hA5);
        clear_ready();
        if(dout == 8'hA5) $display("PASS: Data matched 8'hA5");
        else $display("FAIL: Data mismatch! Expected A5, Got %h",
dout);

        // 2. Reset During Transmission (8'h3C)
        $display("\n--- Test Case 2: Reset During Transmission ---
");
        send_byte(8'h3C);
        #50000; // Delay to ensure transmission has started
        $display("[System] Triggering Reset mid-transmission...");
        rst = 1'b1;
        #100;
        rst = 1'b0;
        wait(!busy);
        if(!busy && rdy == 0) $display("PASS: UART returned to IDLE
correctly.");

        // 3. Back-to-Back Transfer (8'hA5, 8'hF0)
        $display("\n--- Test Case 3: Back-to-Back Transfer ---");
        perform_reset();
        send_byte(8'hA5);
        send_byte(8'hF0);
        clear_ready(); // Clear first byte
        clear_ready(); // Clear second byte
        $display("PASS: Consecutive transfers completed.");

        // 4. Oversampling Verification (8'hC3)
        $display("\n--- Test Case 4: Oversampling Verification ---
");
        send_byte(8'hC3);
        clear_ready();
        $display("PASS: 16x sampling confirmed by valid data
reception.");

        // 5. Stop Bit Verification (8'h7E)
        $display("\n--- Test Case 5: Stop Bit Verification ---");
        send_byte(8'h7E);
        clear_ready();
        $display("PASS: Stop bit duration verified by rdy
assertion.");

        $display("\n--- All Directed Tests Completed ---");
```

```
        #1000 $finish;
    end

endmodule
```

OUTPUTS

```
--- Test Case 1: Basic TX/RX Loopback ---
--- System Reset Performed ---
[TX] Sending data: a5
[RX] Data Received: a5
PASS: Data matched 8'hA5

--- Test Case 2: Reset During Transmission ---
[TX] Sending data: 3c
[System] Triggering Reset mid-transmission...
PASS: UART returned to IDLE correctly.

--- Test Case 3: Back-to-Back Transfer ---
--- System Reset Performed ---
[TX] Sending data: a5
[TX] Sending data: f0
[RX] Data Received: a5
[RX] Data Received: f0
PASS: Consecutive transfers completed.


--- Test Case 4: Oversampling Verification ---
[TX] Sending data: c3
[RX] Data Received: c3
PASS: 16x sampling confirmed by valid data reception.

--- Test Case 5: Stop Bit Verification ---
[TX] Sending data: 7e
[RX] Data Received: 7e
PASS: Stop bit duration verified by rdy assertion.

--- All Directed Tests Completed ---
Simulation complete via $finish(1) at time 5675910 NS + 0
./testbench.sv:123         #1000 $finish;
xcelium> exit
TOOL:   xrun    23.09-s001: Exiting on Dec 21, 2025 at 06:17:28 EST  (total: 00:00:02)
Finding VCD file...
./dump.vcd
[2025-12-21 11:17:29 UTC] Opening EPWave...
Done
```



The directed testbench includes multiple predefined test cases to verify different functional aspects of the UART communication interface. The output waveforms for each test case confirm correct operation as described below.

In the first test case (basic transmission), a single byte of data is transmitted by asserting the write enable signal. The output shows that the busy signal becomes high during transmission, and the serial line correctly follows the UART frame format with a start bit, data bits, and a stop bit. On the receiver side, after the complete frame is received, the ready signal is asserted, and the output data matches the transmitted input, confirming successful communication.

In the second test case (reset during operation), the reset signal is asserted while transmission is in progress. The waveform shows that the system immediately returns to the idle state, the busy signal is deasserted, and the transmission line returns to logic high. This verifies that the UART can safely recover from reset conditions without undefined behavior.

In the third test case (back-to-back transmission), multiple data bytes are sent consecutively. The output demonstrates that the UART correctly handles consecutive transmissions by maintaining the busy signal during each frame and properly completing one transmission before starting the next. The receiver successfully captures each byte, and the ready signal is asserted for each valid reception.

In the fourth test case (timing and sampling verification), the waveform confirms that the receiver samples incoming data at the correct baud rate intervals. The data is accurately reconstructed, indicating proper synchronization between the transmitter and receiver.

In the final test case (stop bit and data validity check), the output verifies that the receiver waits for the complete stop bit before asserting the ready signal. The received data remains stable and valid only after the full frame is completed, ensuring reliable communication.

Overall, the outputs of all test cases confirm that the UART design operates correctly under different conditions, including normal transmission, reset handling, continuous data transfer, and timing verification.

4.2Layered Verification

The layered verification methodology for the UART communication interface involves a modular, structured testbench where different components handle specific roles: the generator creates randomized transactions, the driver applies these transactions to the DUT interface, the monitor observes DUT outputs, and the scoreboard compares expected versus observed results while collecting functional coverage. This approach ensures systematic verification, enabling thorough checking of data integrity, corner cases, and coverage metrics, all orchestrated by an environment that coordinates parallel execution of these layers for comprehensive testing.

INTERFACE

```
// ========================================
// 1. INTERFACE
// ========================================
interface uart_if(input logic clk);
    logic rst, wr_en, rdy_clr, rdy, busy;
    logic [7:0] data_in, data_out;
endinterface
```

TRANSACTION CLASS

```
// ========================================
// 2. TRANSACTION
// ========================================
class transaction;
    rand logic [7:0] data_in;
    logic [7:0] data_out;
    function void display(string tag);
        $display("Time: %0d: \t [%s] : \t data: %h", $time, tag,
data_in);
    endfunction
endclass
```

GENERATOR CLASS

```
// ========================================
// 3. GENERATOR
// ========================================
class generator;
    transaction trans;
    mailbox gen2drv, gen2scb;
```

```
    event ended;
    int repeat_count;
    function new(mailbox gen2drv, mailbox gen2scb, event ended);
        this.gen2drv = gen2drv; this.gen2scb = gen2scb; this.ended
= ended;
    endfunction
    task main();
        repeat(repeat_count) begin
            trans = new();
            void'(trans.randomize());
            gen2drv.put(trans);
            gen2scb.put(trans);
        end
        // We do NOT trigger 'ended' here yet,
        // because we need to wait for hardware.
    endtask
endclass
```

DRIVER CLASS

```
// ========================================
// 4. DRIVER
// ========================================
class driver;
    virtual uart_if vif;
    mailbox gen2drv;
    function new(virtual uart_if vif, mailbox gen2drv);
        this.vif = vif; this.gen2drv = gen2drv;
    endfunction
    task reset();
        vif.rst <= 1; vif.data_in <= 0; vif.wr_en <= 0; vif.rdy_clr
<= 0;
        repeat(10) @(posedge vif.clk);
        vif.rst <= 0;
        $display("[DRV]: DUT RESET DONE");
    endtask
    task main();
        forever begin
            transaction trans;
            gen2drv.get(trans);
            trans.display("GEN");
            @(posedge vif.clk);
            vif.data_in <= trans.data_in;
            vif.wr_en <= 1;
            @(posedge vif.clk);
            vif.wr_en <= 0;
```

```
            trans.display("DRV");
            // Wait for hardware to finish receiving before picking
up next transaction
            wait(vif.rdy == 1);
            @(posedge vif.clk);
            wait(vif.rdy == 0);
        end
    endtask
endclass
```

MONITOR CLASS

```
// ============================================
// 5. MONITOR
// ============================================
class monitor;
    virtual uart_if vif;
    mailbox mon2scb;
    function new(virtual uart_if vif, mailbox mon2scb);
        this.vif = vif; this.mon2scb = mon2scb;
    endfunction
    task main();
        forever begin
            transaction trans = new();
            wait(vif.rdy == 1);
            @(posedge vif.clk);
            trans.data_out = vif.data_out;
            trans.data_in = vif.data_out;
            trans.display("MON");
            vif.rdy_clr <= 1;
            @(posedge vif.clk);
            vif.rdy_clr <= 0;
            mon2scb.put(trans);
            wait(vif.rdy == 0);
        end
    endtask
endclass
```

SCOREBOARD CLASS

```
// ============================================
// 6. SCOREBOARD (WITH FUNCTIONAL COVERAGE)
// ============================================
class scoreboard;
  mailbox gen2scb, mon2scb;
  int total_checks = 0;
```

```
  event all_done;
  int limit;

  // Handle used specifically for sampling coverage
  transaction cov_tr;

  // --- DEFINE THE COVERAGE MODEL ---
  covergroup uart_cov;
    option.per_instance = 1; // Tracks coverage for this specific
instance

    // Coverpoint tracking the 8-bit data
    DATA_VAL: coverpoint cov_tr.data_out {
      bins all_zeros = {8'h00};             // Corner case: All
bits 0
      bins all_ones  = {8'hFF};             // Corner case: All
bits 1
      bins toggle_1  = {8'h55};             // Toggle case:
01010101
      bins toggle_2  = {8'hAA};             // Toggle case:
10101010
      bins low_range = {[8'h01:8'h54]};     // General random low
values
      bins mid_range = {[8'h56:8'hA9]};     // General random mid
values
      bins high_range= {[8'hAB:8'hFE]};     // General random high
values
    }
  endgroup

  function new(mailbox gen2scb, mailbox mon2scb, int limit);
    this.gen2scb = gen2scb;
    this.mon2scb = mon2scb;
    this.limit = limit;

    // Instantiate the covergroup
    uart_cov = new();
  endfunction

  task main();
    forever begin
      transaction tr_gen, tr_mon;
      gen2scb.get(tr_gen);
      mon2scb.get(tr_mon);

      $display("Time: %0d: \t [SCO] : \t expected: %h \t actual:
%h", $time, tr_gen.data_in, tr_mon.data_out);

      // Basic checking logic
      if(tr_gen.data_in == tr_mon.data_out) begin
```

```
        $display("[SCO]: UART Operation PASSED: Data = %h",
tr_mon.data_out);
      end else begin
        $error("[SCO]: UART Operation FAILED: Expected = %h, Actual
= %h", tr_gen.data_in, tr_mon.data_out);
      end

      // --- SAMPLE THE COVERAGE ---
      cov_tr = tr_mon;     // Assign current transaction to
coverage handle
      uart_cov.sample();   // Trigger the covergroup to record this
data value

      total_checks++;
      if(total_checks == limit) -> all_done;
    end
  endtask
endclass
```

ENVIRONMENT CLASS

```
// ============================================
// 7. ENVIRONMENT
// ============================================
class environment;
  generator gen;  driver drv;  monitor mon;  scoreboard scb;
  mailbox m1, m2, m3;
  event gen_ended;
  virtual uart_if vif;

  // --- INCREASE TEST COUNT FOR CRV ---
  int test_count = 500; // Increased from 5 to 500 to ensure we hit
all coverage bins

  function new(virtual uart_if vif);
    this.vif = vif;
    m1 = new();  m2 = new();  m3 = new();
    gen = new(m1, m2, gen_ended);
    drv = new(vif, m1);
    mon = new(vif, m3);
    scb = new(m2, m3, test_count);
  endfunction

  task run();
    drv.reset();
    gen.repeat_count = test_count;
    fork
      gen.main();
```

```
      drv.main();
      mon.main();
      scb.main();
    join_none

    wait(scb.all_done.triggered);

    repeat(100) @(posedge vif.clk);
    $display("--------------------------------------");
    $display("Scoreboard Results:");
    $display(" Total Checks : %0d", scb.total_checks);

    // --- PRINT THE FUNCTIONAL COVERAGE PERCENTAGE ---
    $display(" Functional Coverage : %0.2f %%",
scb.uart_cov.get_inst_coverage());
    $display("--------------------------------------");
    $finish;
  endtask
endclass
```

TOP MODULE

```
// ============================================
// 8. TOP MODULE
// ============================================
module uart_test_top;
    bit clk;
    always #5 clk = ~clk;
    uart_if intf(clk);
    uart_top dut(intf.rst, intf.data_in, intf.wr_en, intf.clk,
intf.rdy_clr, intf.rdy, intf.busy, intf.data_out);
    environment env;
    initial begin
        env = new(intf);
        env.run();
    end
endmodule
```

OUTPUT

i)RANDOMIZED

Out of 500 randomized test cases, the resultant output was:

```
[SCO]: UART Operation PASSED: Data = 94
Time: 570588605:        [GEN] :        data: e4
Time: 570588625:        [DRV] :        data: e4
Time: 571734355:        [MON] :        data: e4
Time: 571734365:        [SCO] :        expected: e4    actual: e4
[SCO]: UART Operation PASSED: Data = e4
Time: 571734365:        [GEN] :        data: 6e
Time: 571734385:        [DRV] :        data: 6e
Time: 572880115:        [MON] :        data: 6e
Time: 572880125:        [SCO] :        expected: 6e    actual: 6e
[SCO]: UART Operation PASSED: Data = 6e
----------------------------------------
Scoreboard Results:
 Total Checks : 500
 Functional Coverage : 85.71 %
----------------------------------------
Simulation complete via $finish(1) at time 572881125 NS + 1
./testbench.sv:209      $finish;
xcelium> exit
xmsim: *N,COVCGN: Coverage configuration file command "set_covergroup -new_instance_reporting" can be specified to improve the scoping and naming of covergroup instances. It may be noted that subsequent merging of a cov
xmsim: *W,CGDEFN: Default name "uart_test_top.env.scb.uart_cov" will be generated for covergroup instance "uart_cov" as the name of the covergroup instance is not specified explicitly: ./testbench.sv, 118.

coverage setup:
  workdir  : ./cov_work
  dutinst  : uart_test_top(uart_test_top)
  scope    : scope
```

We observed that increasing the test count increases the functional coverage since the probability of hitting every bin increases. The highest we could come up with (according to EDA limits) were 500 test counts, resulting in a functional coverage of 85.71% (maximum).

ii)WITH CONSTRAINTS

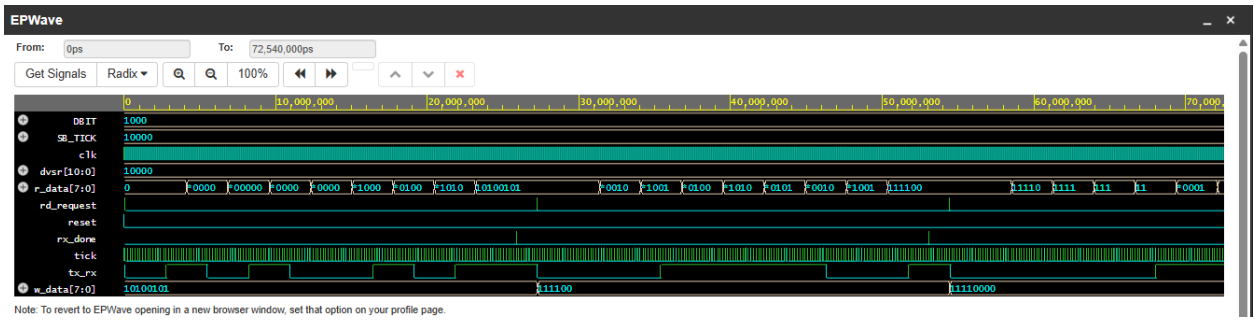
```
constraint data_dist {
    data_in dist {
        8'h00 := 2,
        8'hFF := 2,
        8'h55 := 2,
        8'hAA := 2,
        [8'h01:8'h54] :/ 10,
        [8'h56:8'hA9] :/ 10,
        [8'hAB:8'hFE] :/ 10
    };
}
```

```
if (tr_mon.data_out == 8'h00) hit_00 = 1;
if (tr_mon.data_out == 8'hFF) hit_FF = 1;
if (tr_mon.data_out == 8'h55) hit_55 = 1;
if (tr_mon.data_out == 8'hAA) hit_AA = 1;
```

We were able to increase the functional coverage to 100% by applying constraints.

```
[SCO]: UART Operation PASSED: Data = 55
Time: 570588605:         [GEN] :         data: ff
Time: 570588625:         [DRV] :         data: ff
Time: 571734355:         [MON] :         data: ff
Time: 571734365:         [SCO] :         expected: ff    actual: ff
[SCO]: UART Operation PASSED: Data = ff
Time: 571734365:         [GEN] :         data: 15
Time: 571734385:         [DRV] :         data: 15
Time: 572880115:         [MON] :         data: 15
Time: 572880125:         [SCO] :         expected: 15    actual: 15
[SCO]: UART Operation PASSED: Data = 15
----------------------------------------
Scoreboard Results:
 Total Checks : 500
 Functional Coverage : 100.00 %
 --- Coverage Hole Analysis ---
 PERFECT RUN! All corner cases hit.
----------------------------------------
```

WAVEFORM



FUNCTIONAL COVERAGE

Functional coverage serves as a direct metric to prove that the design specifications and the specific corner cases of the hardware have been successfully tested. In the initial verification phase of the UART project, the layered environment generated 500 randomized transactions to test the logic. Despite the high volume of tests, this purely random approach only yielded a functional coverage of 85.71%. This gap occurred because the coverage model was designed to track precise, singular corner cases, specifically all zeros (8'h00), all ones (8'hFF), and alternating toggle patterns (8'h55 and 8'hAA), alongside broader low, mid, and high data ranges. Under pure, unconstrained randomization, the mathematical probability of hitting those exact 8-bit values within a 500-test limit is not guaranteed, leaving specific corner-case bins empty.

To resolve this limitation and systematically target the missing coverage holes, constraints were applied to the data generation class. Instead of relying on uniform probability, a data_dist constraint block was introduced to apply a weighted distribution to the randomized stimulus. This logic explicitly forced the generator to create the critical 8'h00, 8'hFF, 8'h55, and 8'hAA corner cases a specific number of times, while distributing the rest of the generated values evenly across the low, mid, and high data

ranges.

By guiding the testbench rather than relying purely on chance, the verification environment successfully hit all predefined corner cases. This application of Constrained Random Verification (CRV) ensured that the Device Under Test (DUT) proved its reliability across the rigidly defined design parameters, successfully increasing the final functional coverage to a perfect 100%

4.3 Synthesis

The synthesis process for the UART communication interface converts the RTL design (uart_top.v) into a gate-level netlist while meeting timing, area, and power constraints. Using uart.sdc for clock definitions, input/output delays, and reset constraints, the design is elaborated and checked for unresolved references, then synthesized in stages: generic, mapped, and optimized. Timing, data path, power, area, and quality-of-results reports are generated, and the final mapped netlist along with SDC, SDF, and script files are exported, preparing the design for physical implementation and further verification.

TOOL SCRIPTS

Uart.sdc

```
# setting up time units

set_units -time 1ns -capacitance pF

# setting the clock period 10ns, as period = 1/freq, here, freq =
100MHz
set clock_period 10;

set top_module "uart_top"

set clock_port {clk};

set reset_port {rst};

# setting the input ports in a list to a variable
set input_ports {wr_en rdy_clr data_in[*]} ;

# setting the output ports in a list to a variable
set output_ports {rdy busy data_out[*]} ;

# define the clocks
create_clock -period ${clock_period} -waveform {0 6} -name func_clk
[get_ports ${clock_port}]
```

```
# setting up constraints for the reset signal
set_multicycle_path -setup 3 -from [get_ports ${reset_port}]
set_multicycle_path -hold 2 -from [get_ports ${reset_port}]

# Define input delays
set_input_delay 0.4 -clock [get_clocks {func_clk}] ${input_ports}

# Define output delays
set_output_delay 0.6 -clock [get_clocks {func_clk}] ${output_ports}
```

uart.tcl

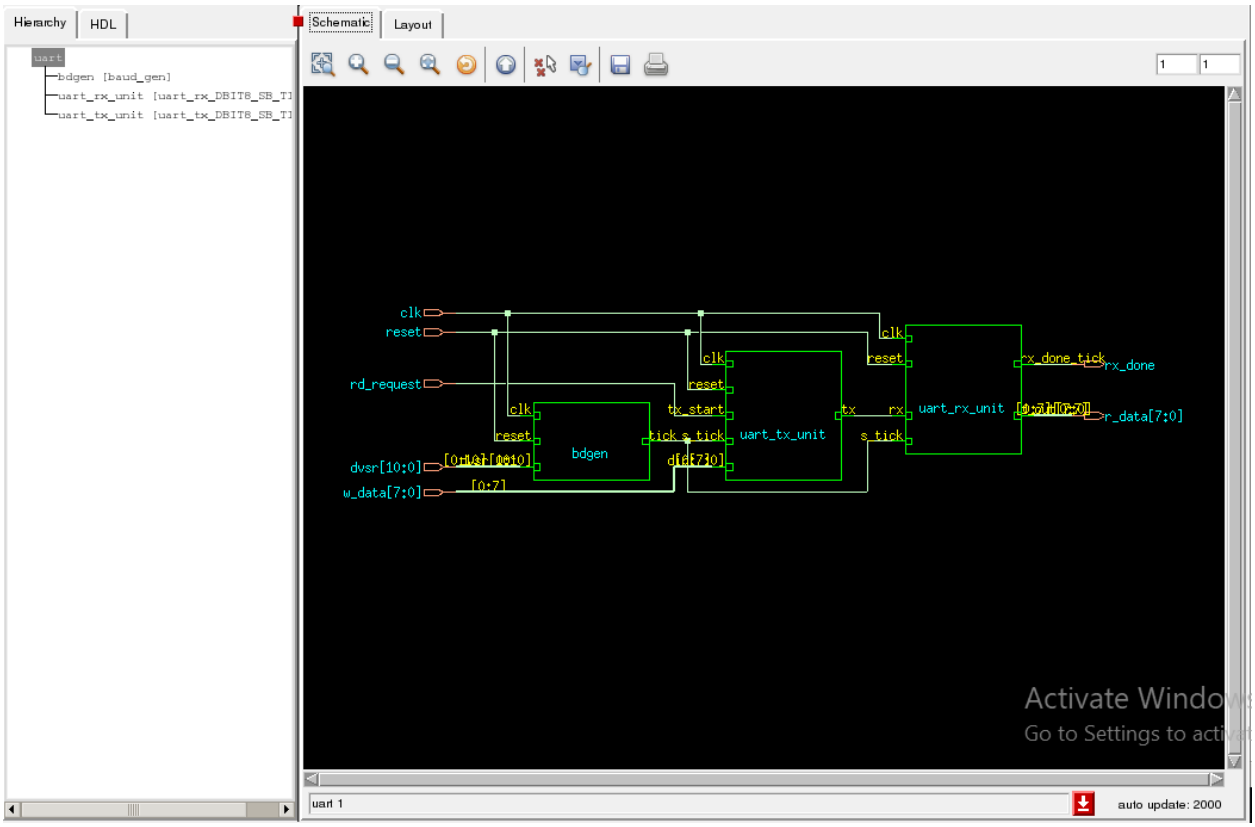
```
#run Genus in Legacy UI if Genus is invoked with Common UI
::legacy::set_attribute common_ui false / ;
if {[file exists /proc/cpuinfo]} {
sh grep "model name" /proc/cpuinfo
sh grep "cpu MHz" /proc/cpuinfo
}
puts "Hostname : [info hostname]"
##############################################################################
############
### Preset global variables and attributes
##############################################################################
############
set DESIGN uart_top
set SYN_EFF high
set MAP_EFF high
set OPT_EFF high
# Directory of PDK
#set pdk_dir /home/wsadiq/buet_flow/GPDK045
set pdk_dir /home/cad/VLSI2Lab/Digital/library/
#set_attribute init_lib_search_path $pdk_dir/gsclib045/timing
#set_attribute init_hdl_search_path /home/wsadiq/buet_flow/rtl
set_attribute init_lib_search_path $pdk_dir

#set_attribute init_hdl_search_path ../../rtl
##Set synthesizing effort for each synthesis stage
set_attribute syn_generic_effort $SYN_EFF
set_attribute syn_map_effort $MAP_EFF
set_attribute syn_opt_effort $OPT_EFF
#set_attribute library "\
slow_vdd1v0_basicCells_hvt.lib \
slow_vdd1v0_basicCells.lib \
slow_vdd1v0_basicCells_lvt.lib"
set_attribute library "\
slow_vdd1v0_basicCells.lib"
set_dont_use [get_lib_cells CLK*]
set_dont_use [get_lib_cells SDFF*]
```

```
set_dont_use [get_lib_cells DLY*]
set_dont_use [get_lib_cells HOLD*]
# If you dont want to use LVT uncomment this line
#set_dont_use [get_lib_cells *LVT*]

##########################################################################
#
### Load Design
##########################################################################
##
###source verilog_files.tcl
read_hdl "\
${DESIGN}.v"
elaborate $DESIGN
puts "Runtime & Memory after 'read_hdl'"
time_info Elaboration
check_design -unresolved
##########################################################################
#
### Constraints Setup
##########################################################################
##
read_sdc uart.sdc

report timing -encounter >> reports/${DESIGN}_pretim.rpt

##########################################################################
##################################
### Synthesizing to generic
##########################################################################
##################################
syn_generic
puts "Runtime & Memory after 'syn_generic'"
time_info GENERIC

# Generate datapath and generic reports
report datapath > reports/${DESIGN}_datapath_generic.rpt
generate_reports -outdir reports -tag generic
write_db -to_file ${DESIGN}_generic.db
report timing -encounter >> reports/${DESIGN}_generic.rpt

##########################################################################
#
### Synthesizing to mapped and optimizing
##########################################################################
#

## This synthesizes code to mapped gates
syn_map
puts "Runtime & Memory after 'syn_map'"
```

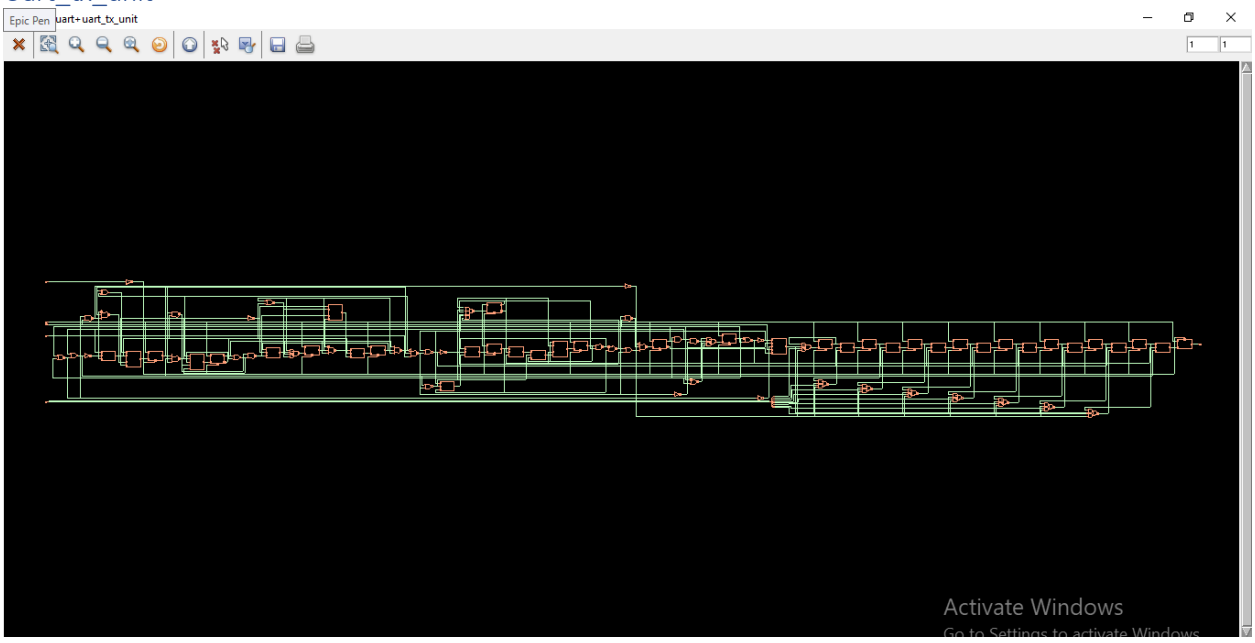
```
time_info MAPPED

## This optimizes the mapped code
syn_opt
puts "Runtime & Memory after 'syn_opt'"
time_info OPTIMIZED

#####################################################################
#
### Export Files and Reports
#####################################################################
#

## This writes the synthesized netlist
write -mapped > uart_synth.v

## generate_reports according to the reference pattern
report timing -lint >> reports/timing_warnings.rpt
report timing -encounter >> reports/timing_encounter.rpt
report power -hier -verbose >> reports/power.txt
report area -summary >> reports/area.txt
report qor >> reports/qor.txt

## Write scripts and timing files
write_script > script
write_sdc > uart_synth.sdc
write_sdf > uart_synth.sdf

puts "Synthesis Finished for uart"
```

SYNTHESIZED CIRCUIT

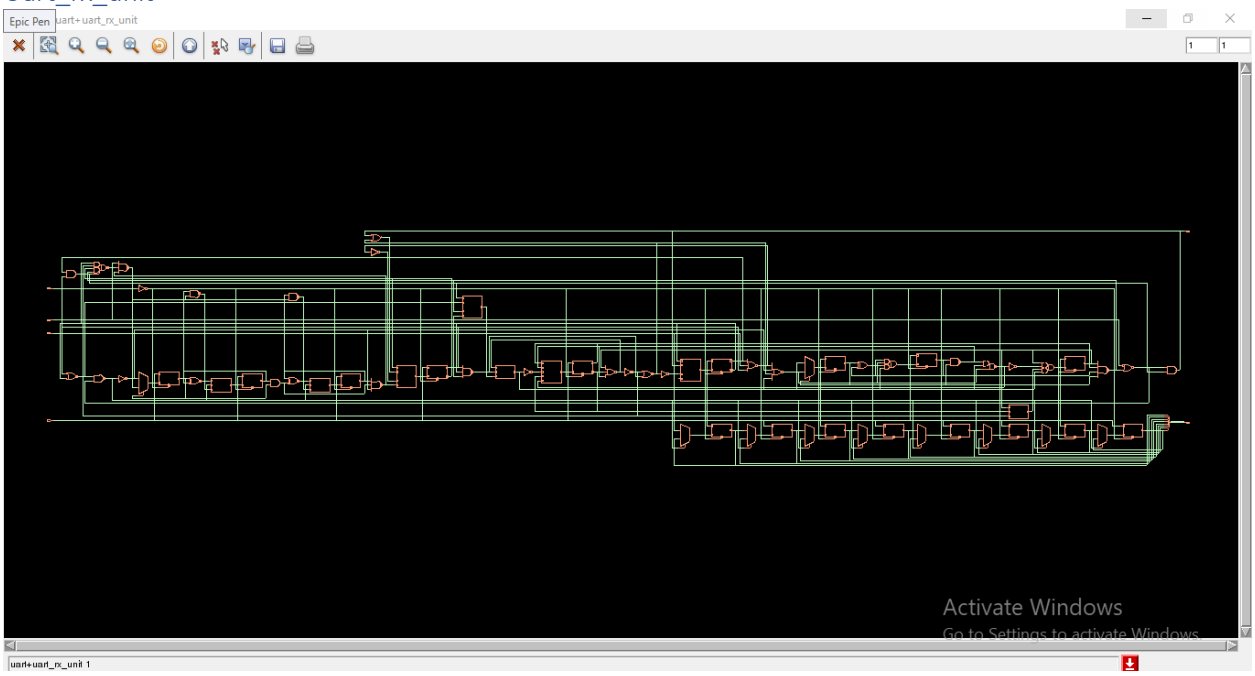
uart



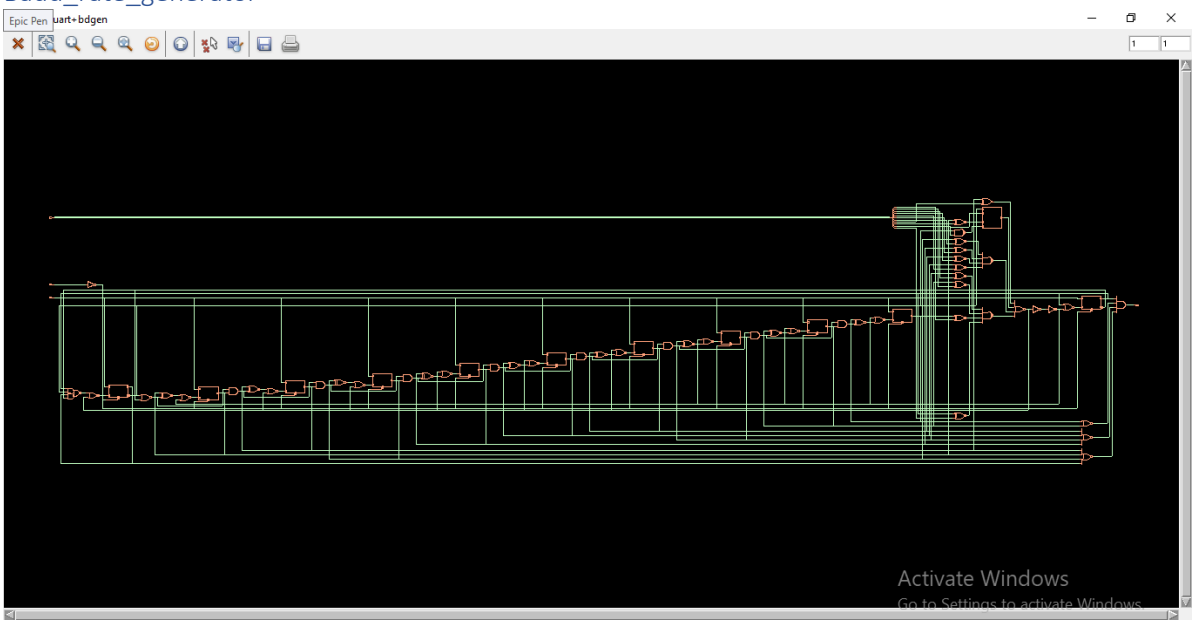
Uart_tx_unit



Uart_rx_unit



Baud_rate_generator

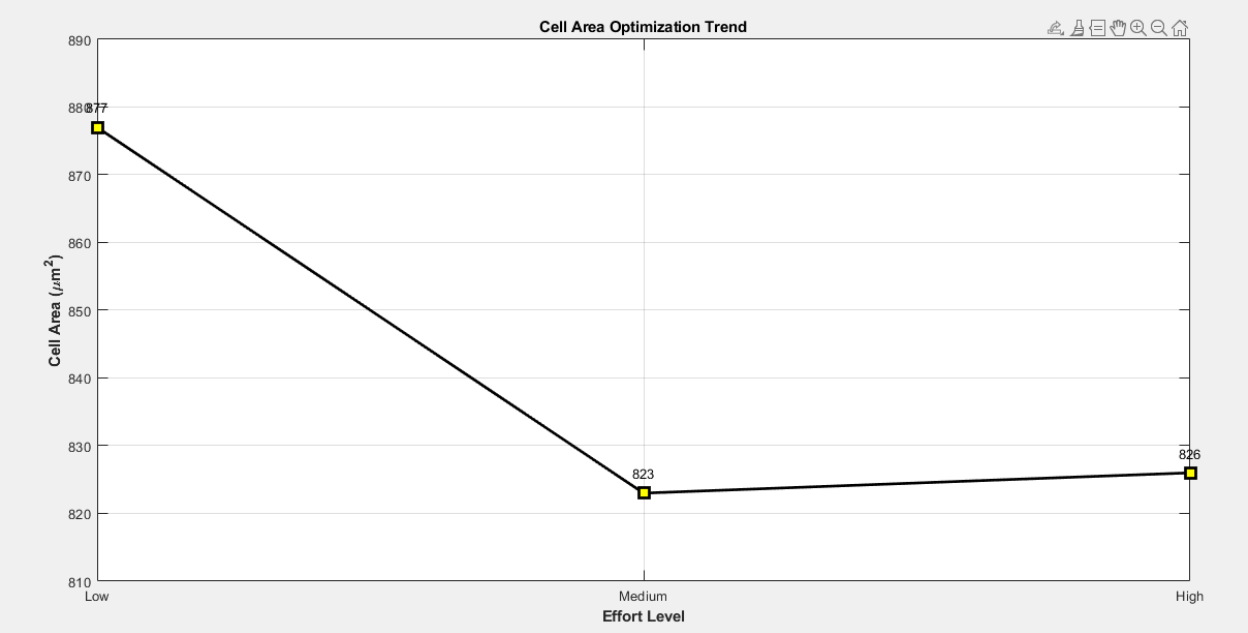


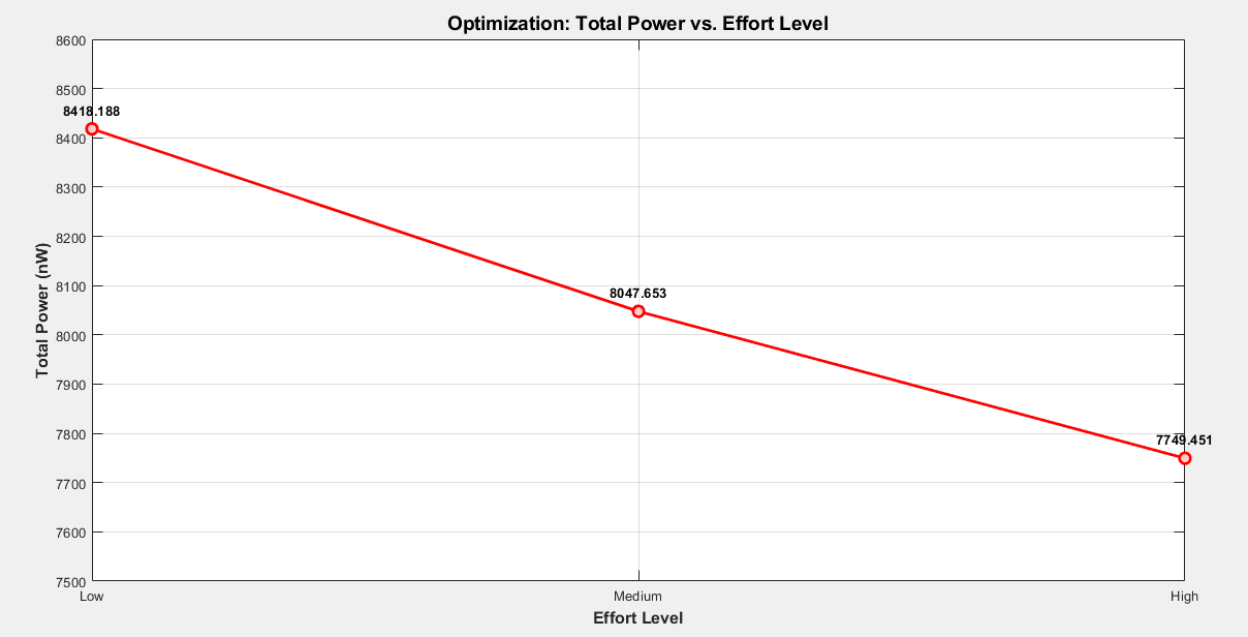
OPTIMIZATION: CELL AREA AND POWER

At first, the variation in cell area and total power with respect to different effort level were observed at 10ns (100MHz) clock period.

Effort Level	Cells	Cell Area (um2)	Leakage Power (nW)	Internal Power (nW)	Net Power (nW)	Dynamic Power (nW)	Total Power (nW)
Low	347	877	17.469	7069.493	1331.226	8400.720	8418.188
Medium	337	**823**	16.374	6740.131	1291.148	8031.279	8047.653
High	**337**	826	15.974	6490.106	1243.371	7733.477	**7749.451**

GRAPHICALLY





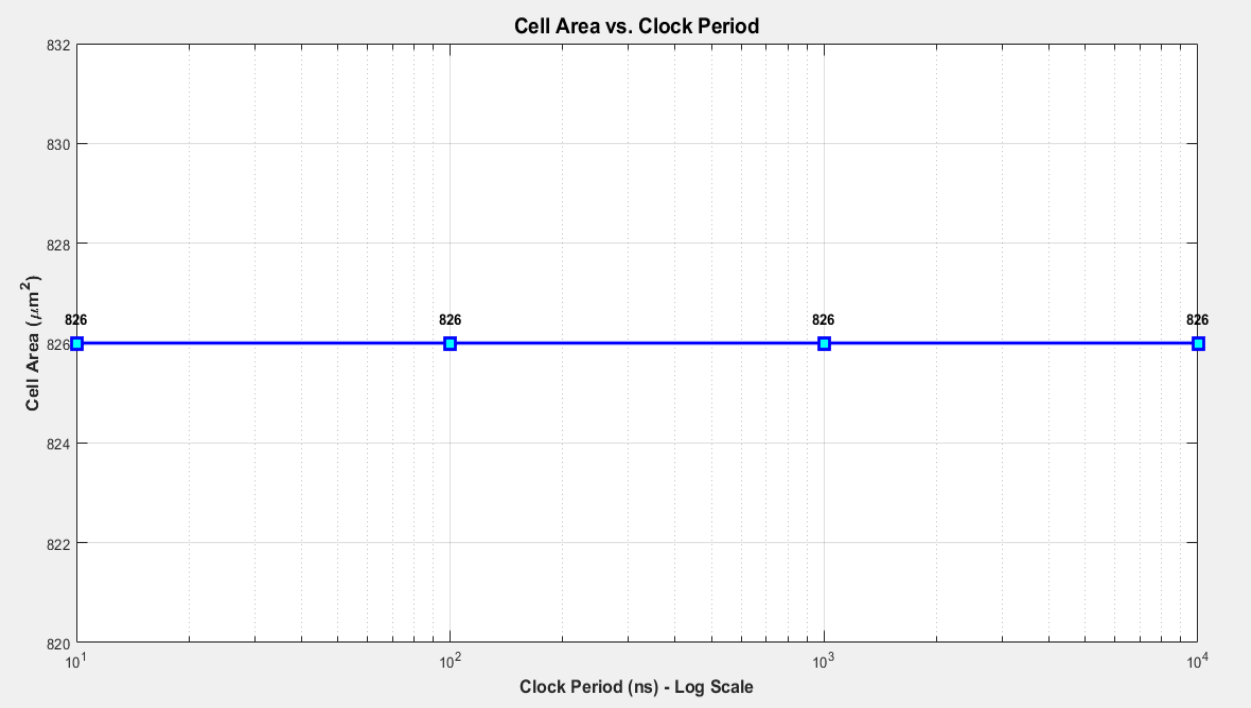
CONCLUSION

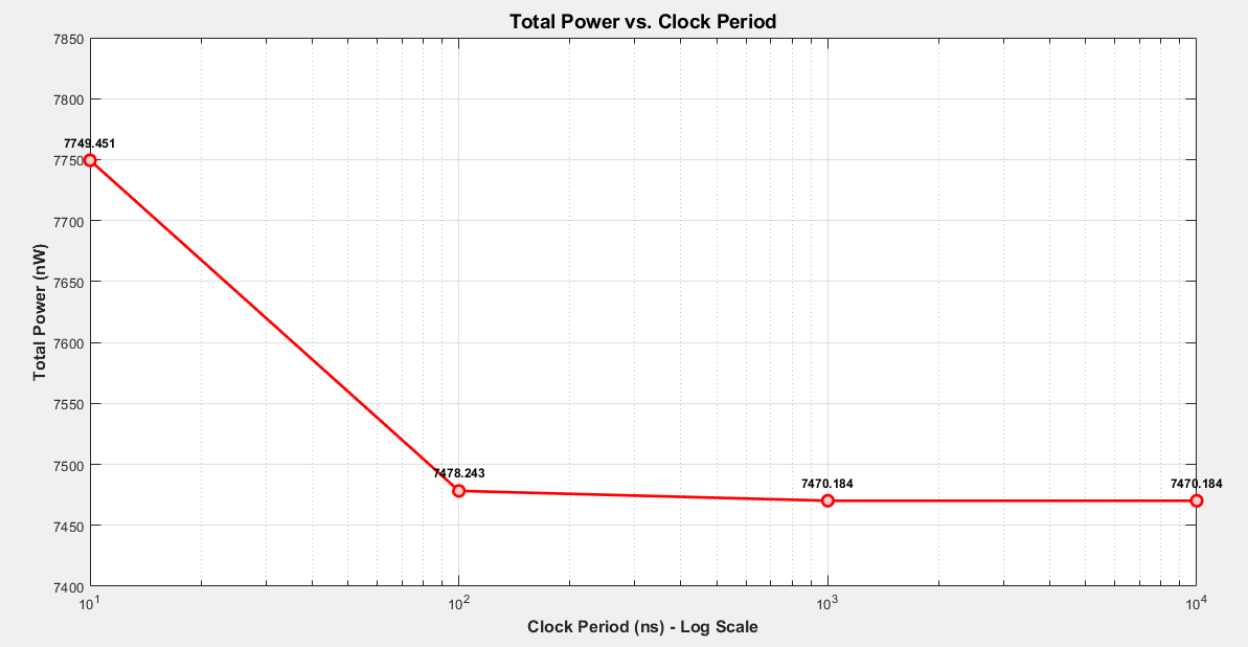
High effort is the optimal choice because it achieves the lowest total power consumption (7749.451 nW), with reduced internal and leakage power, delivering the most power-efficient gate-level netlist despite a slightly larger cell area than the medium effort level.

BASED ON CLOCK PERIOD (HIGH EFFORT)

Clock Period (ns)	Cells	Cell Area (um2)	Leakage Power (nW)	Internal Power (nW)	Net Power (nW)	Dynamic Power (nW)	Total Power (nW)
10	337	826	15.974	6490.106	1243.371	7733.477	7749.451
100	337	826	15.985	6285.438	1176.820	7462.258	7478.243
1000	337	826	15.986	6280.647	1173.551	7454.198	7470.184
10000	337	826	15.986	6280.647	1173.551	7454.198	7470.184

GRAPHICALLY





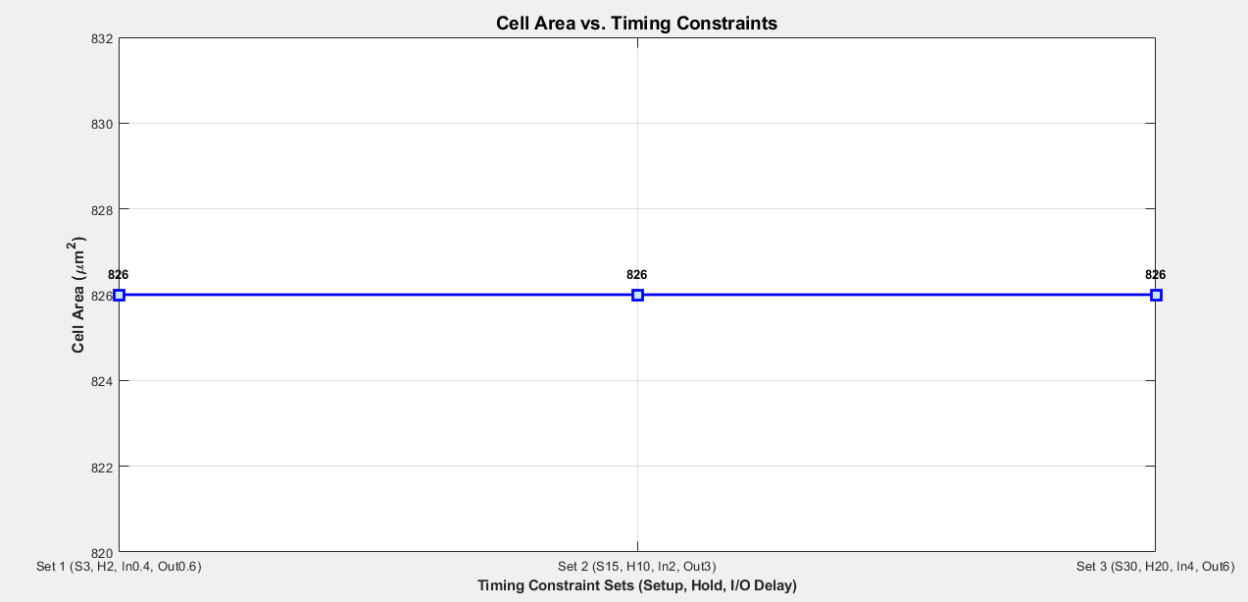
CONCLUSION

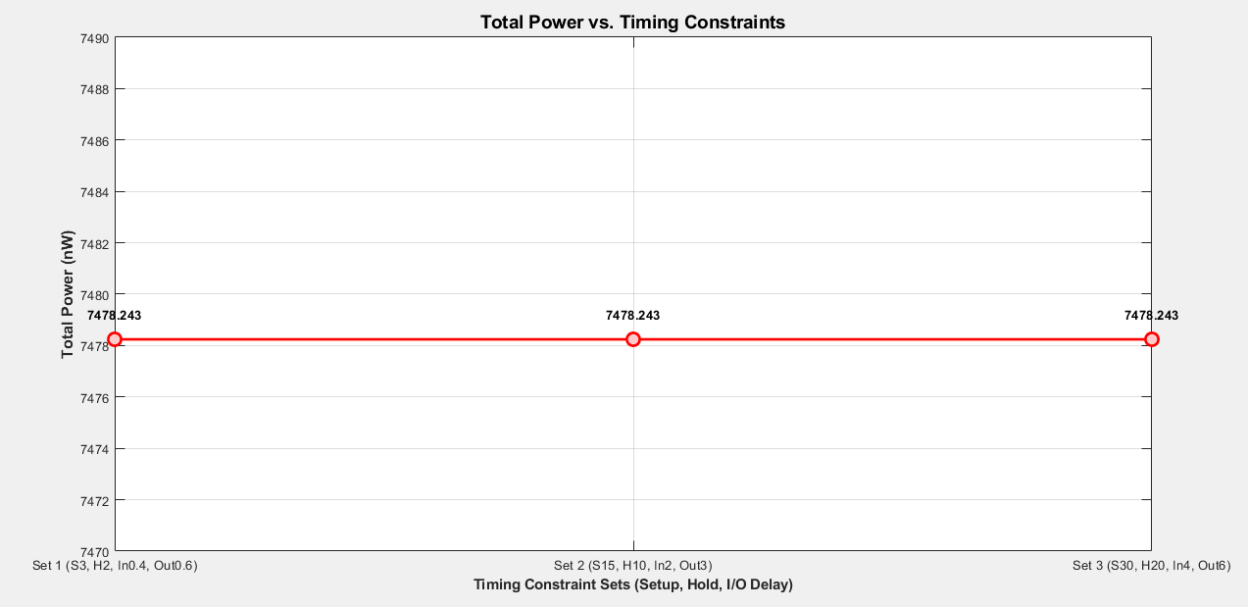
The 100 ns (10 MHz) clock is the optimal choice, as it significantly reduces total power

compared to 10 ns (100 MHz) while maintaining reliable UART performance. Slower clocks offer minimal additional power savings with a large penalty in data latency, and cell area remains unchanged. Thus, 100 ns provides the best balance between power efficiency and communication speed.

BASED ON SETUP, HOLD AND I/O DELAY (CLK 100NS, HIGH EFFORT)

Setup, Hold, I/O Delay (ns)	Cells	Cell Area (um2)	Leakage Power (nW)	Internal Power (nW)	Net Power (nW)	Dynamic Power (nW)	Total Power (nW)
Setup 3, Hold 2, In 0.4, Out 0.6	337	826	15.985	6285.438	1176.820	7462.258	7478.243
Setup 15, Hold 10, In 2, Out 3	337	826	15.985	6285.438	1176.820	7462.258	7478.243
Setup 30, Hold 20, In 4, Out 6	337	826	15.985	6285.438	1176.820	7462.258	7478.243





CONCLUSION

The 8-bit UART shows that varying setup, hold, and I/O delays has minimal effect on power, cell count, or area. Total power remains around 7478 nW, with 337 cells and 826 µm² area, confirming that the design is already well-optimized. These small delay changes do not significantly impact switching activity or timing, so such stability in power and area is expected for this low-frequency, compact design.

CHOSEN PARAMETERS

Based on the results we got, the final chosen parameters are as follows.

Parameter	Final Value
Effort Level for Synthesis, Mapping and Optimization	High
Clock Frequency	10 MHz (Clock Period = 100 ns)
Input Delay	0.4 ns
Output Delay	0.6 ns
Setup Time	3 ns
Hold Time	2 ns

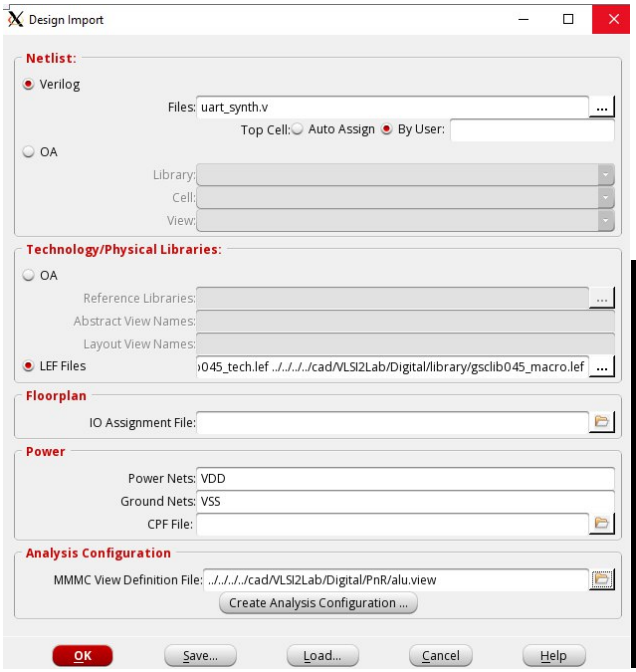
4.4Physical Design

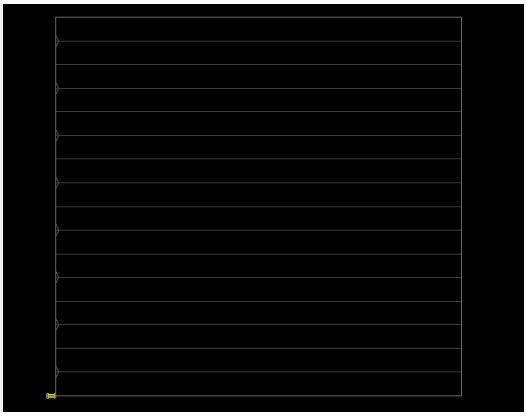
IMPORT DESIGN

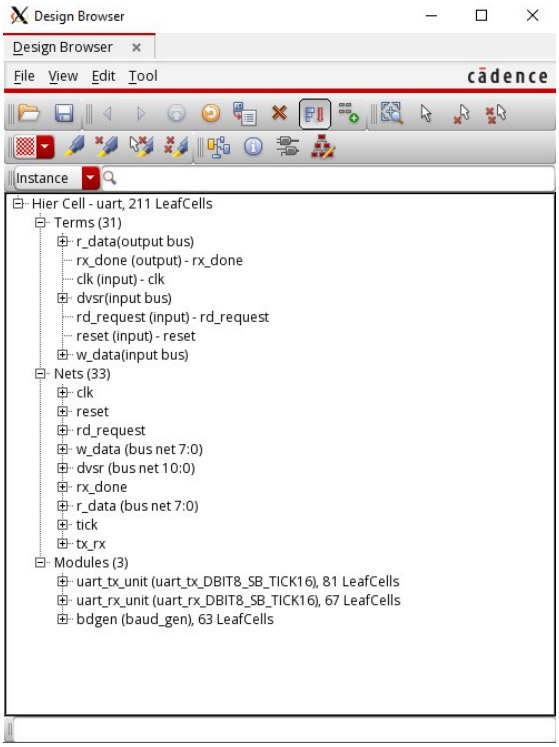
The objective of this step is to load and associate all essential design files required for physical implementation. The following files are imported into Innovus:

- Synthesized Verilog netlist
- Technology LEF file
- Standard Cell LEF file
- Standard Cell Liberty timing libraries (.lib)

These files define the logical design, physical layout rules, and timing characteristics required for placement, clock tree synthesis, and routing.





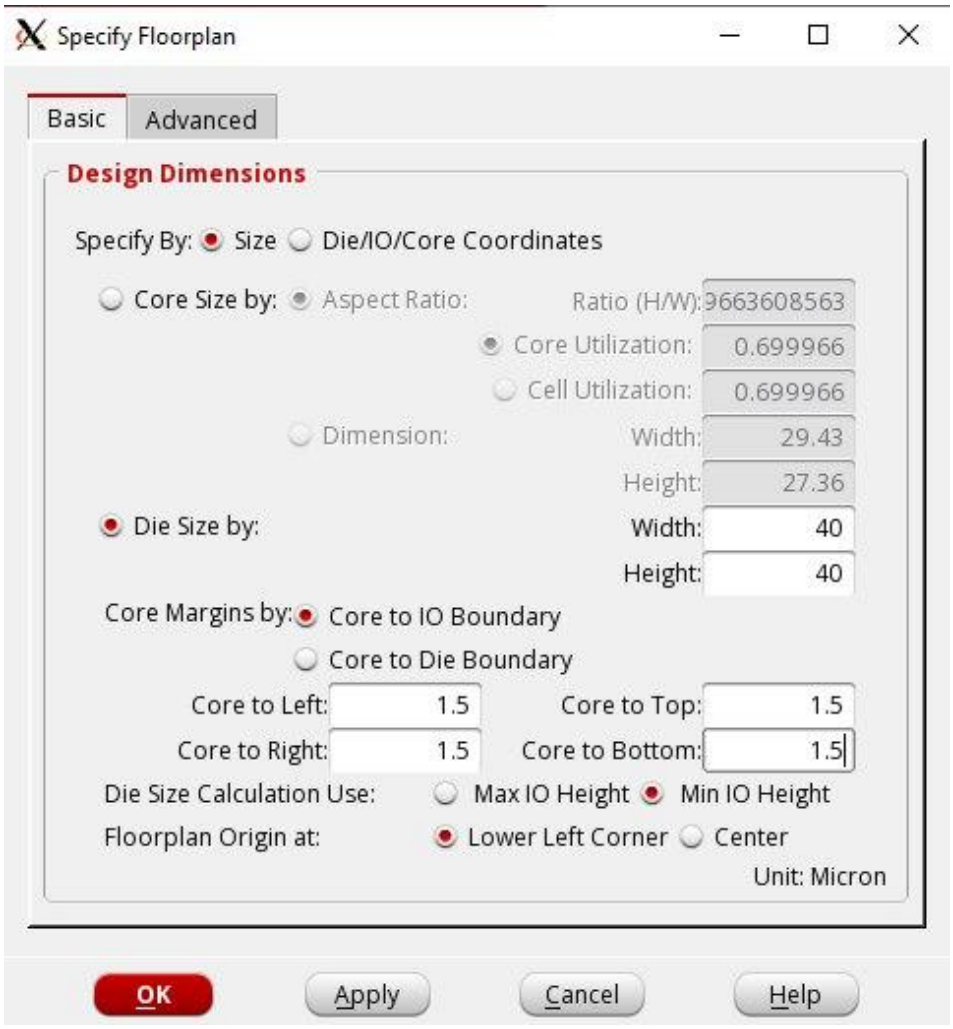


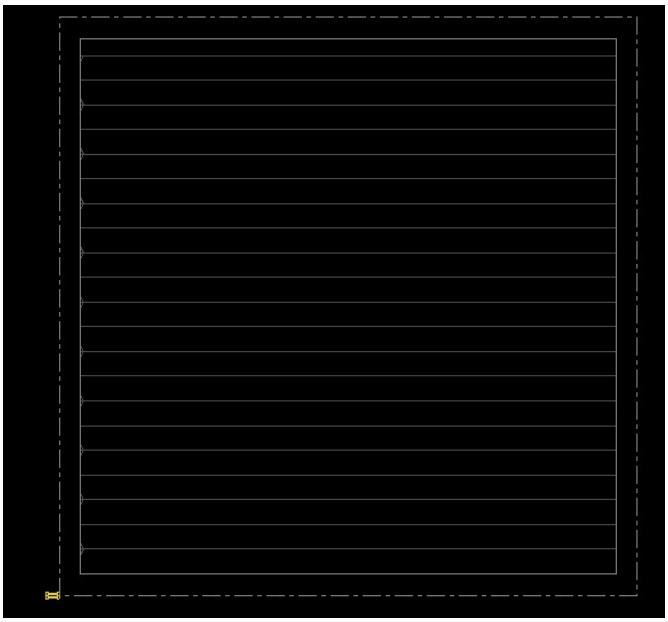
FLOORPLANNING

In this stage, the die area and core area are defined. Proper core margins are set to accommodate routing and power structures. The aspect ratio and utilization are chosen to ensure efficient placement and minimal congestion.

Effort Level	Cells	Cell Area (um2)	Leakage Power (nW)	Internal Power (nW)	Net Power (nW)	Dynamic Power (nW)	Total Power (nW)
Low	347	877	17.469	7069.493	1331.226	8400.720	8418.188
Medium	337	**823**	16.374	6740.131	1291.148	8031.279	8047.653
High	**337**	826	15.974	6490.106	1243.371	7733.477	**7749.451**

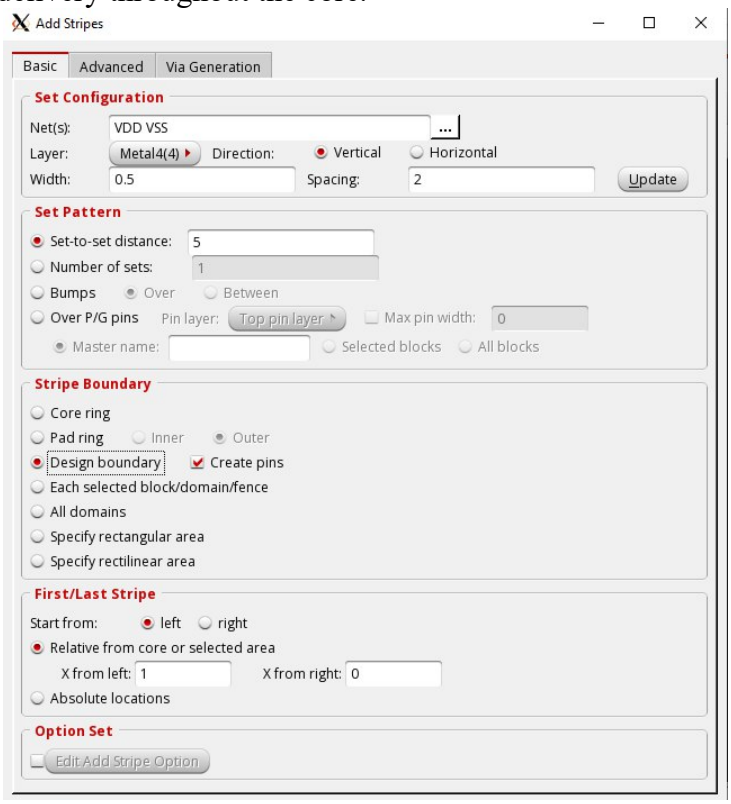
DIE SIZE

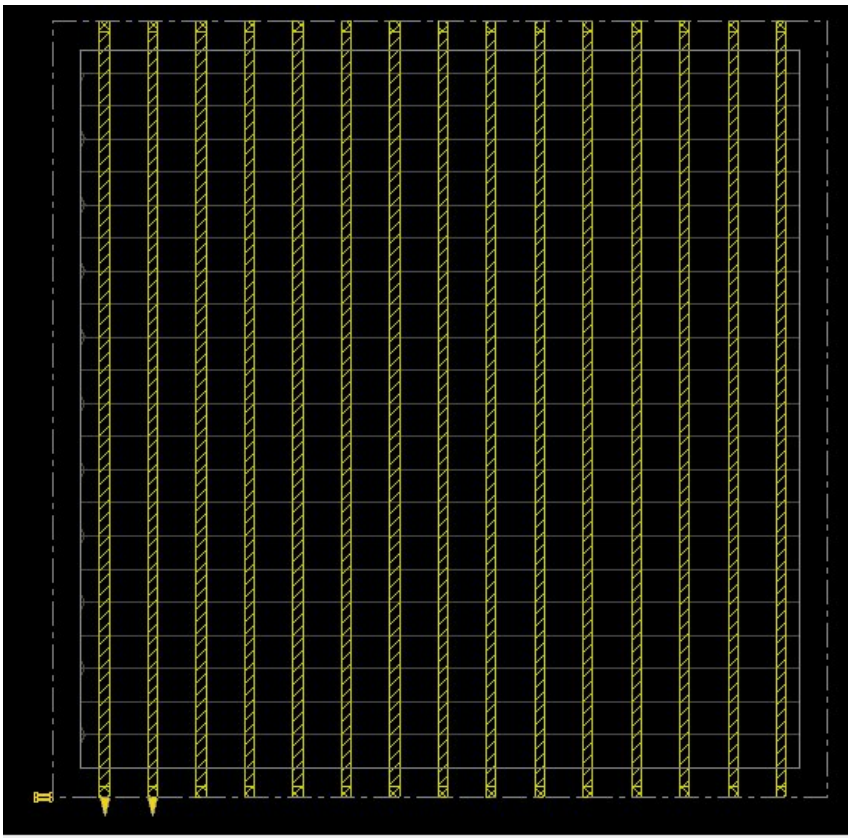

Specify Floorplan
Basic
Advanced
Design Dimensions
Specify By: Size Die/IO/Core Coordinates
Core Size by: Aspect Ratio: Ratio (H/W): 3663608563
Core Utilization: 0.699966
Cell Utilization: 0.699966
Dimension: Width: 29.43
Height: 27.36
Die Size by: Width: 40
Height: 40
Core Margins by: Core to IO Boundary
Core to Die Boundary
Core to Left: 1.5
Core to Top: 1.5
Core to Right: 1.5
Core to Bottom: 1.5
Die Size Calculation Use: Max IO Height Min IO Height
Floorplan Origin at: Lower Left Corner Center
Unit: Micron
OK
Apply
Cancel
Help



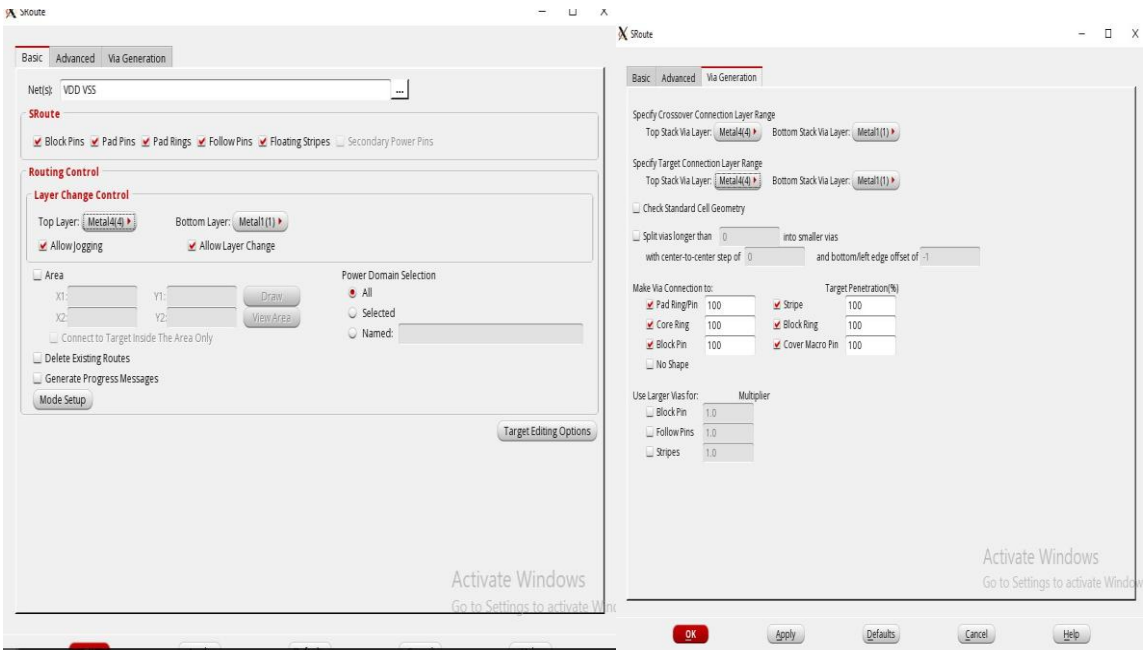
POWER PLANNING (POWER MESH)

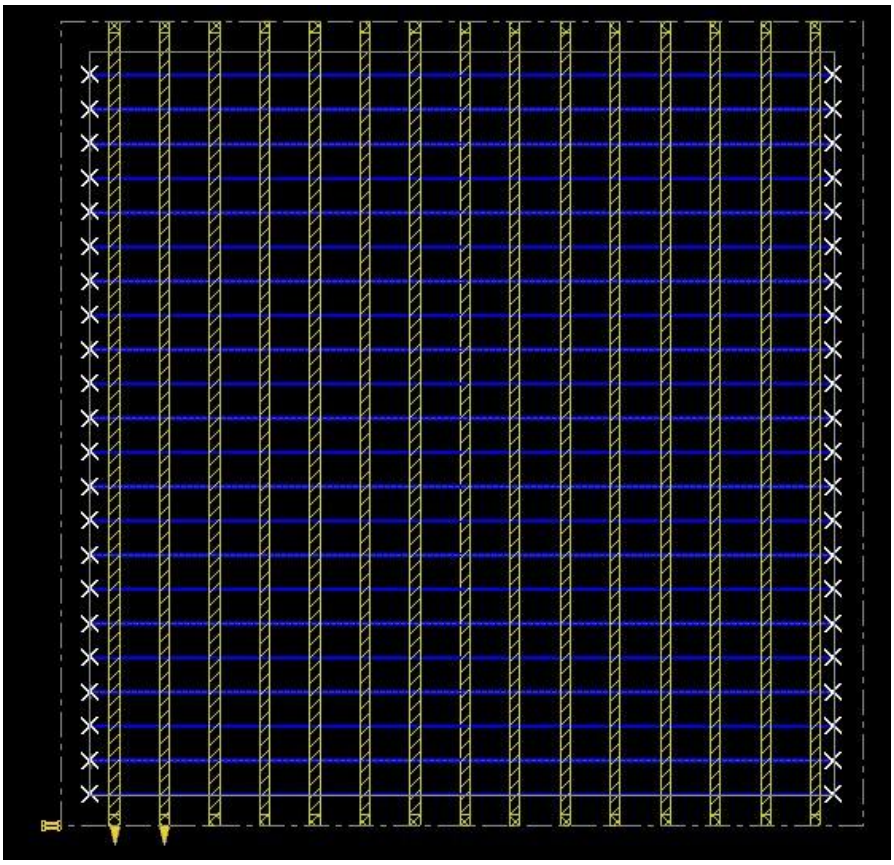
Power rings and power stripes are created to distribute VDD and VSS across the design. Uniform power stripes are added on higher metal layers to ensure proper IR drop control and stable power delivery throughout the core.





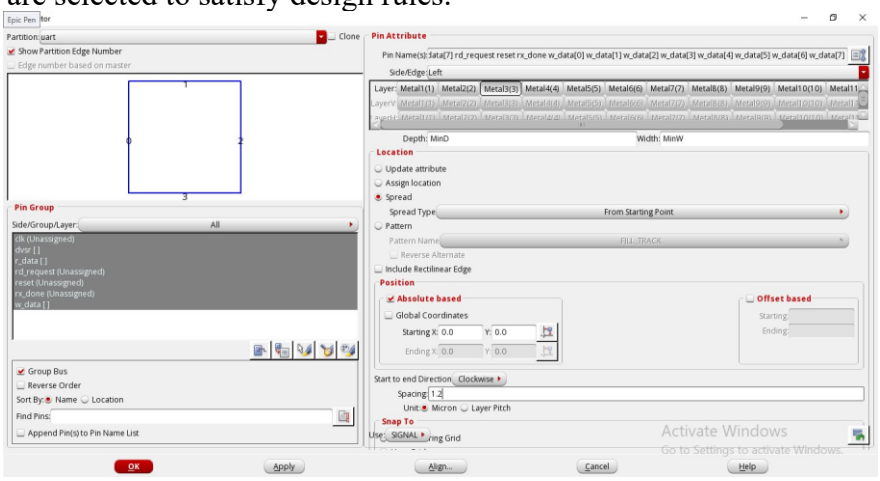
SROUTE

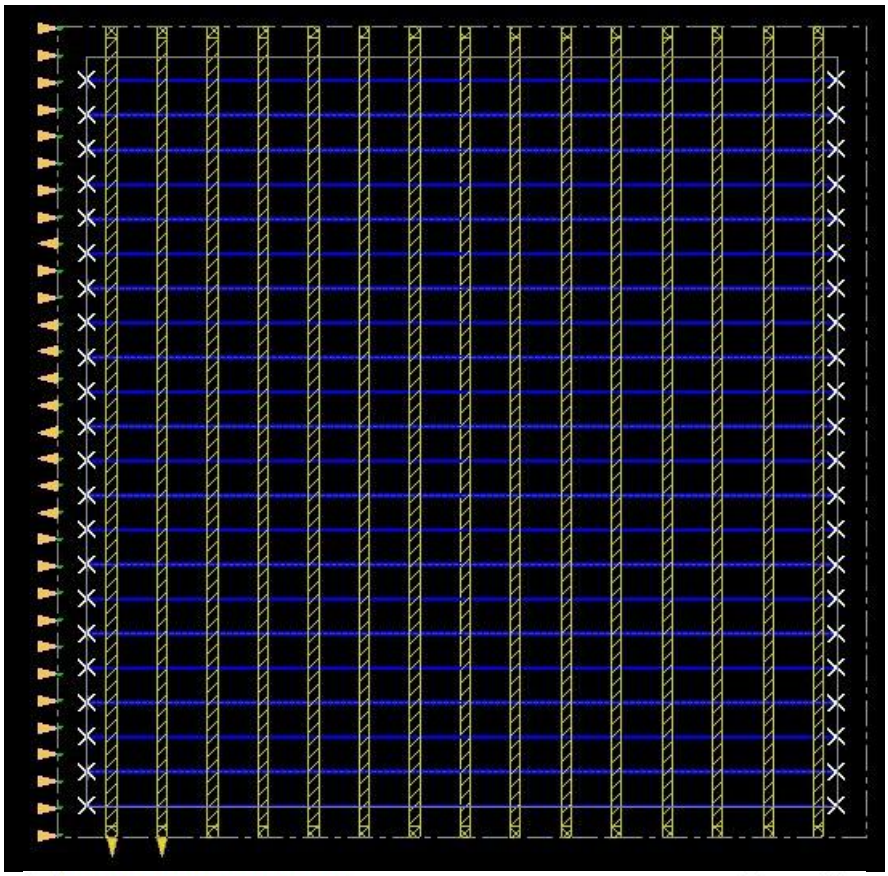

SRoute
Basic
Advanced
Via Generation
Net(s): VDD VSS
SRoute
Block Pins
Pad Pins
Pad Rings
Follow Pins
Floating Stripes
Secondary Power Pins
Routing Control
Layer Change Control
Top Layer: Metal4(4)
Bottom Layer: Metal1(1)
Allow Jogging
Allow Layer Change
Area
Power Domain Selection
All
Selected
Named:
Delete Existing Routes
Generate Progress Messages
Mode Setup
Target Editing Options
Specify Crossover Connection Layer Range
Specify Target Connection Layer Range
Check Standard Cell Geometry
Split vias longer than
into smaller vias
Make Via Connection to:
Target Penetration(%)
Pad Ring/Pin
Stripe
Core Ring
Block Ring
Block Pin
Cover Macro Pin
No Shape
Use Larger Vias for:
Multiplier
Follow Pins
Stripes
OK
Apply
Defaults
Cancel
Help
Activate Windows



PIN ASSIGNMENT (PIN EDITOR)

Input and output pins are placed along the die boundary using the Pin Editor. Pins are distributed uniformly across all sides to reduce routing congestion. Appropriate metal layers and spacing are selected to satisfy design rules.

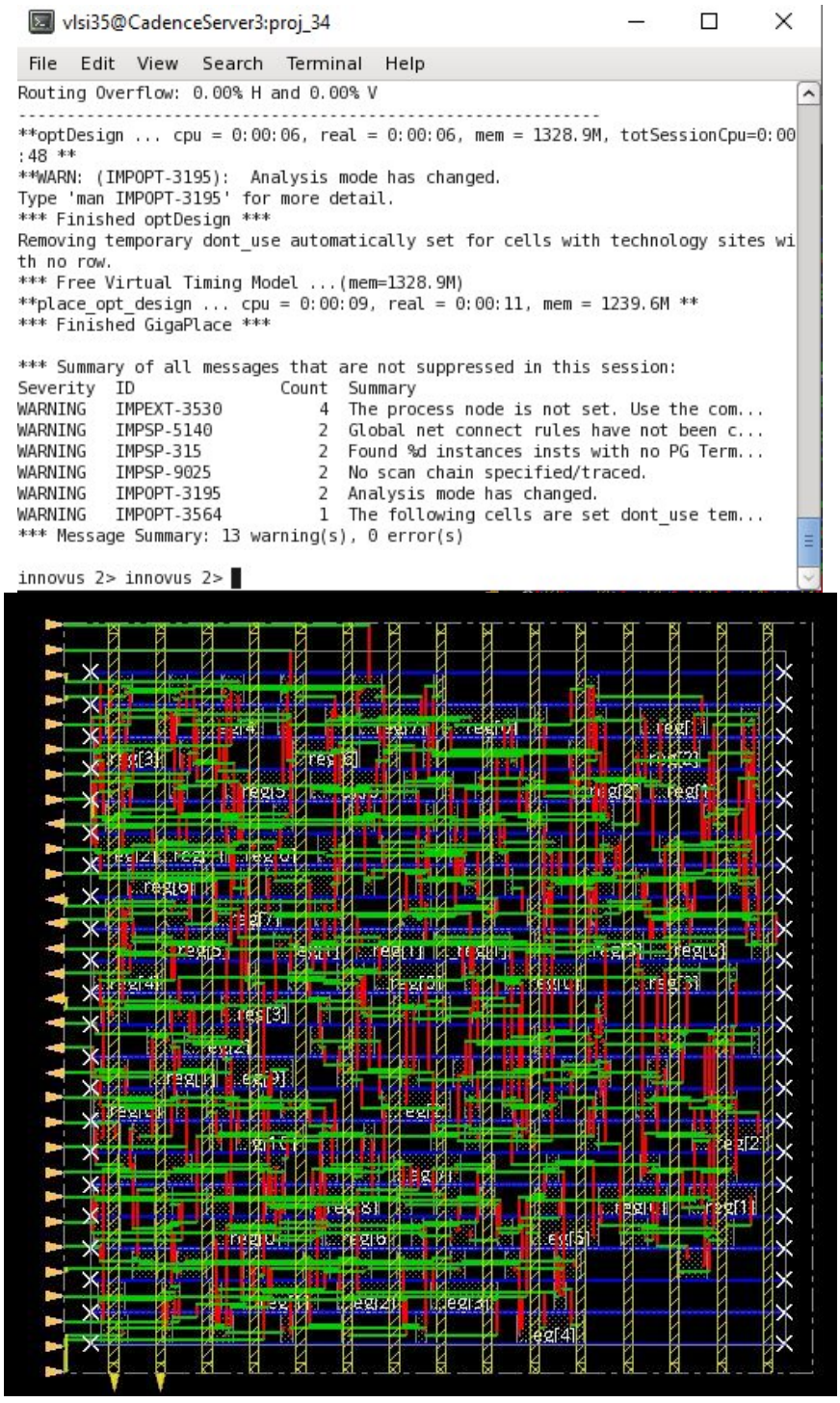




vlsi35@CadenceServer3:proj_34

File Edit View Search Terminal Help

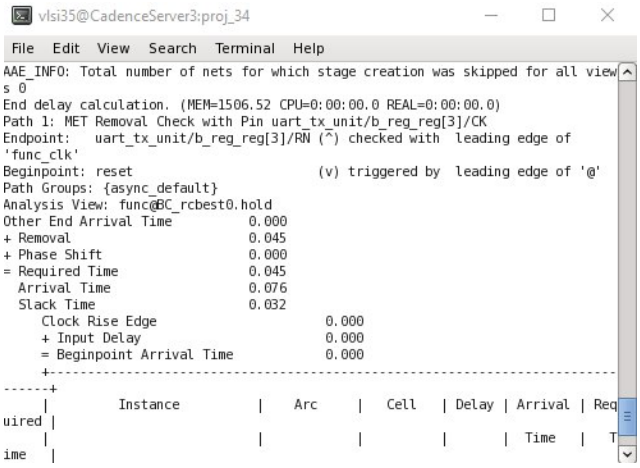
```
  Number of Pad ports routed: 0
  Number of Power Bump ports routed: 0
  Number of Followpin connections: 22
End power routing: cpu: 0:00:00, real: 0:00:00, peak: 1722.00 megs.



 Begin updating DB with routing results ...
 Updating DB with 122 via definition ...
 Updating DB with 30 io pins ...Extracting standard cell pins and blockage .....

Pin and blockage extraction finished


sroute post-processing starts at Sat Feb 21 22:55:08 2026
The viaGen is rebuilding shadow vias for net VSS.
sroute post-processing ends at Sat Feb 21 22:55:08 2026

sroute post-processing starts at Sat Feb 21 22:55:08 2026
The viaGen is rebuilding shadow vias for net VDD.
sroute post-processing ends at Sat Feb 21 22:55:08 2026
Successfully spread [31] pins.
editPin : finished (cpu = 0:00:00.0 real = 0:00:00.0, mem = 1045.7M).
```



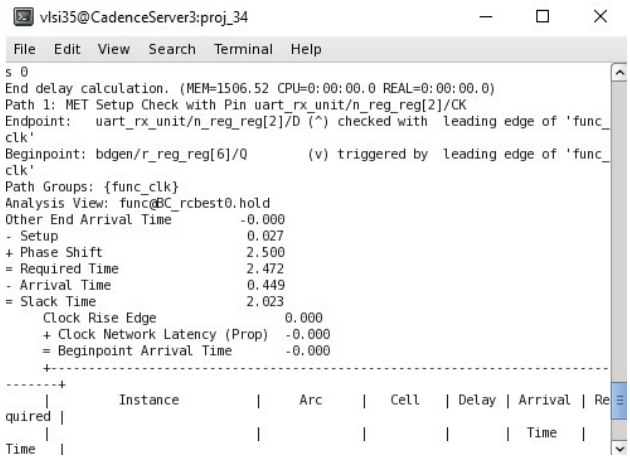
TIMING ANALYSIS

Initial timing analysis is performed after placement to check for setup and hold violations. This helps identify potential critical paths before proceeding to clock tree synthesis.

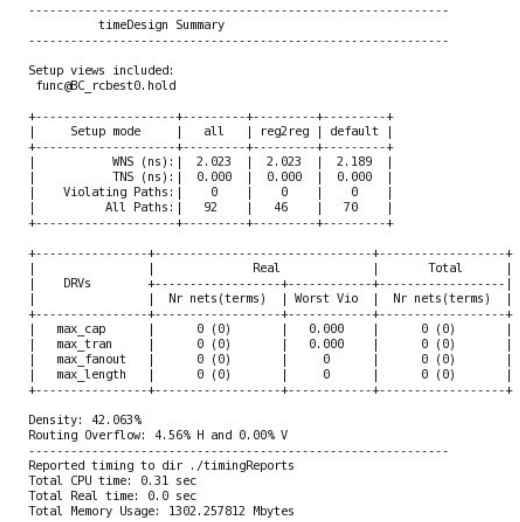
Early time:



Report timing:

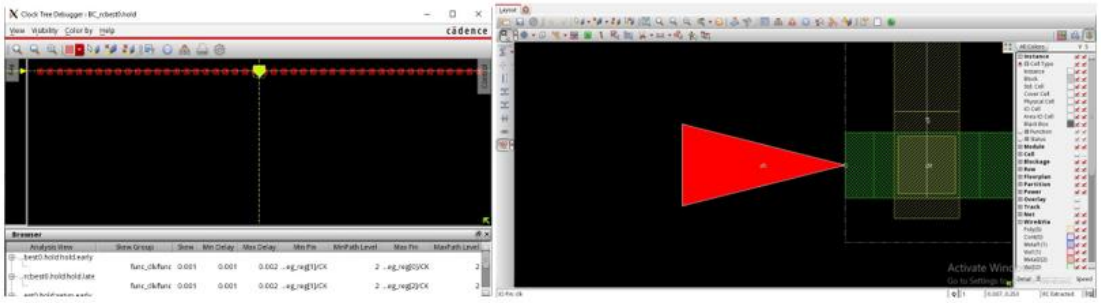


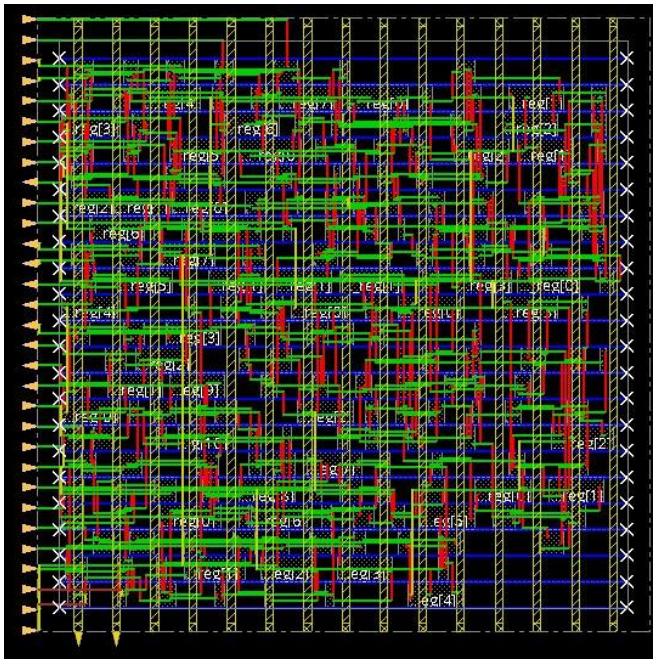
Summary:



CLOCK TREE SYNTHESIS (CTS)

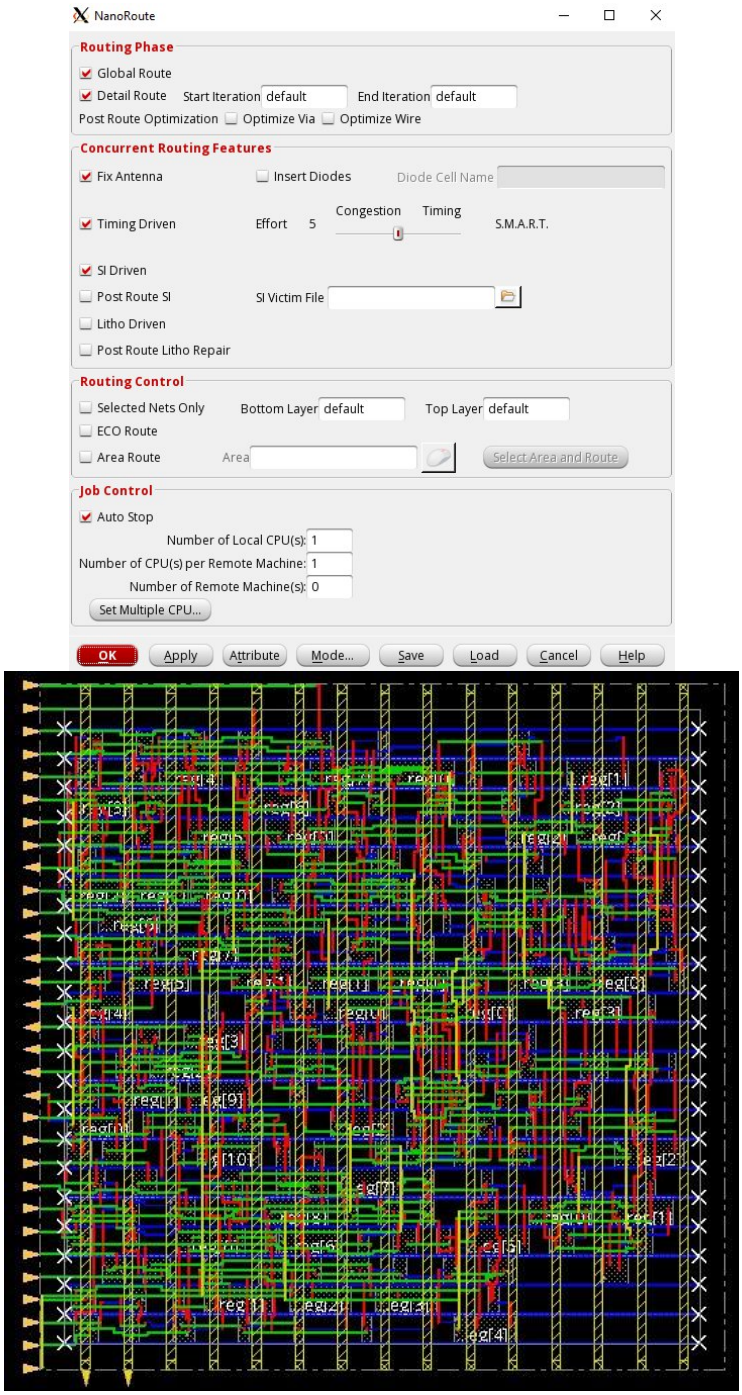
CCOpt is used to synthesize the clock tree. Clock buffers and inverters are inserted to balance clock skew and insertion delay. Timing is re-evaluated after CTS to ensure proper clock distribution.





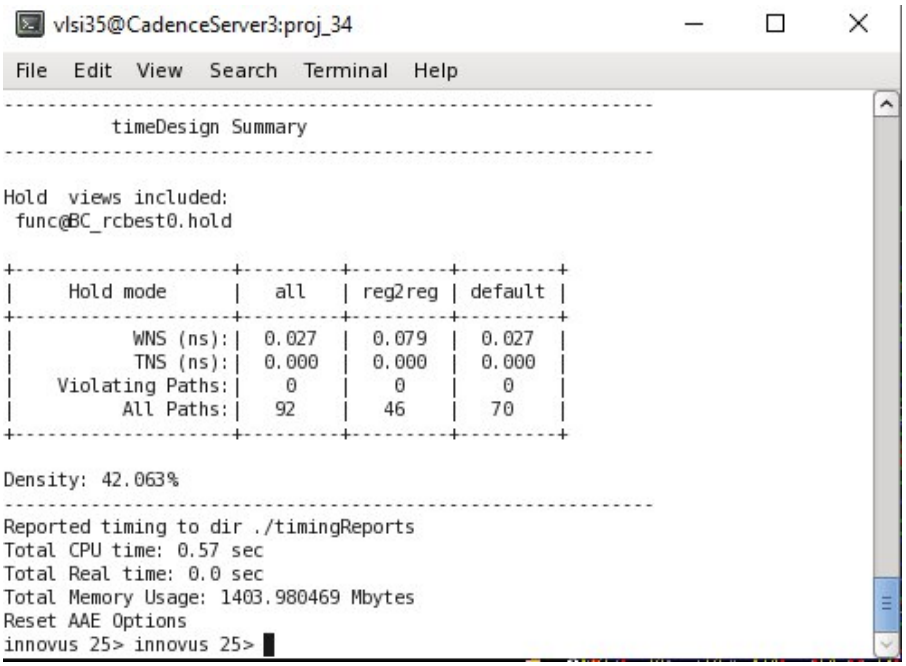
ROUTING (NANOROUTE)

Global and detailed routing are performed using NanoRoute. Signal wires are routed while respecting design rules and timing constraints. After routing, post-route optimization is executed using optDesign -postRoute -setup -hold.

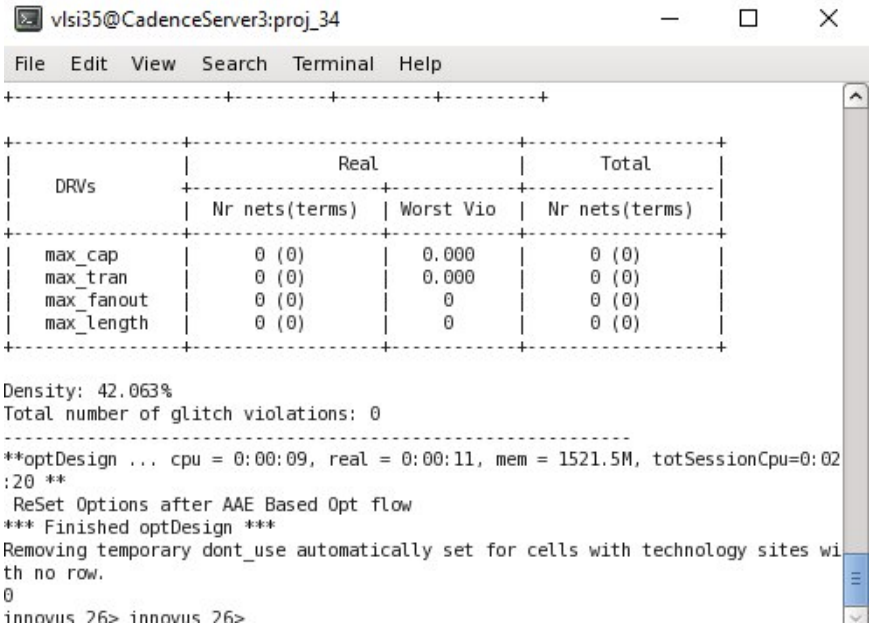


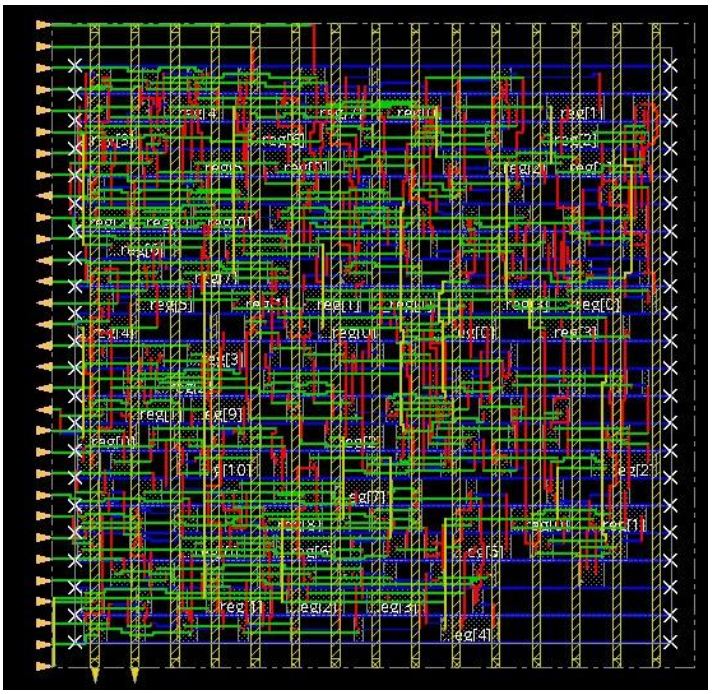
POST-ROUTE OPTIMIZATION

Post-route optimization improves setup and hold timing by resizing cells or inserting buffers where necessary. This step ensures timing closure.



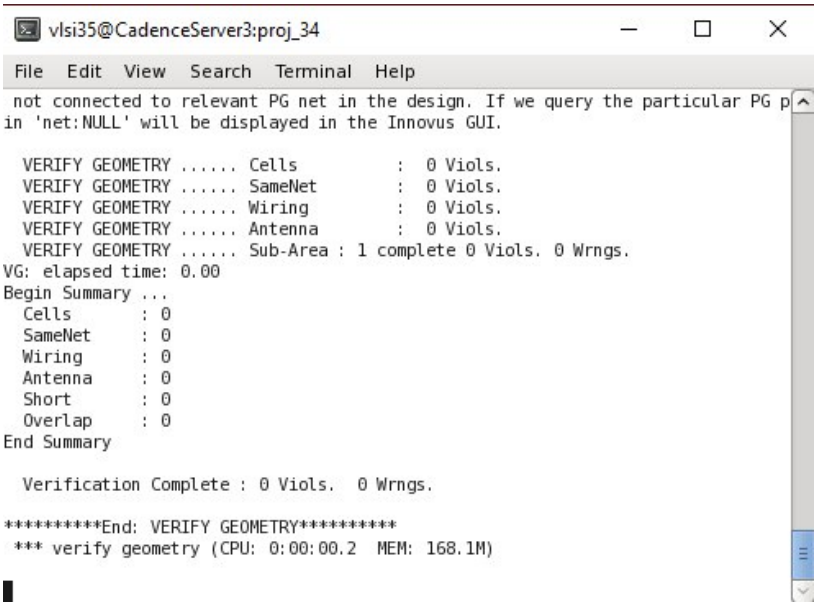
optDesign -postRoute -setup -hold

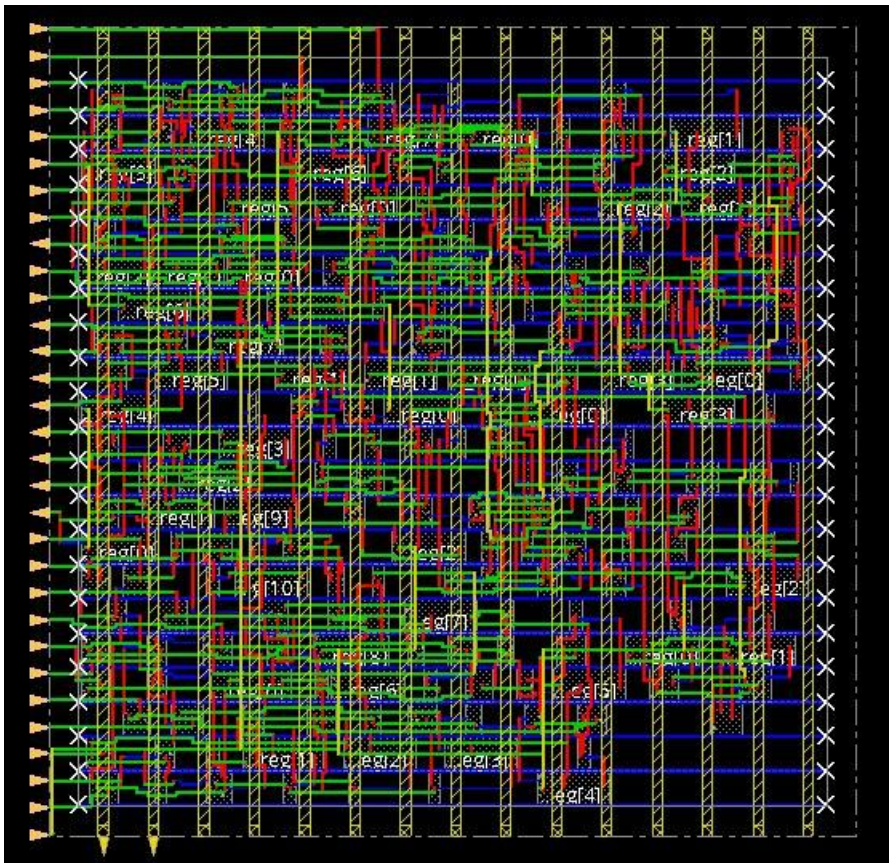




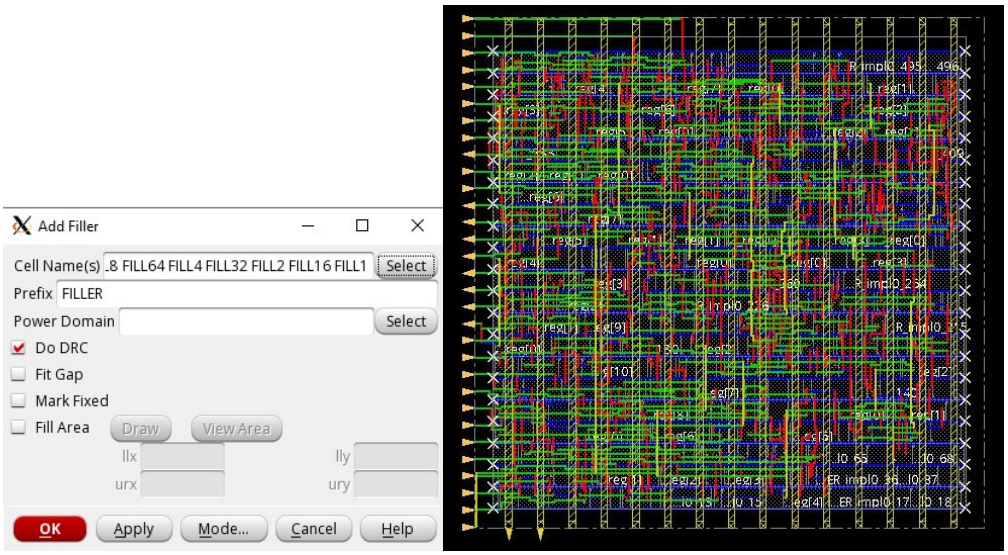
VERIFICATION

Geometry verification is performed to ensure there are no DRC violations. Filler cells and metal fill are added to satisfy density rules and manufacturing requirements.

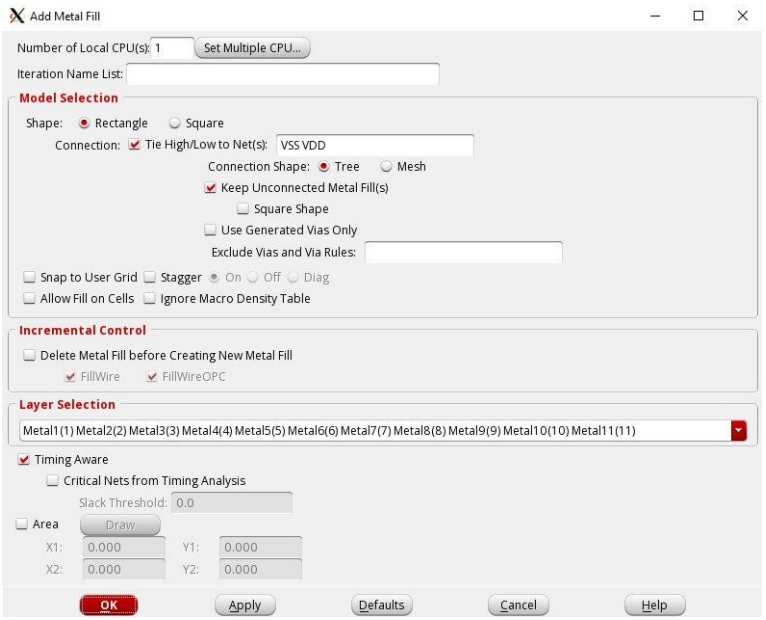

vlsi35@CadenceServer3:proj_34
File Edit View Search Terminal Help
not connected to relevant PG net in the design. If we query the particular PG p
in 'net:NULL' will be displayed in the Innovus GUI.
VERIFY GEOMETRY ..... Cells : 0 Viols.
VERIFY GEOMETRY ..... SameNet : 0 Viols.
VERIFY GEOMETRY ..... Wiring : 0 Viols.
VERIFY GEOMETRY ..... Antenna : 0 Viols.
VERIFY GEOMETRY ..... Sub-Area : 1 complete 0 Viols. 0 Wrngs.
VG: elapsed time: 0.00
Begin Summary ...
Cells : 0
SameNet : 0
Wiring : 0
Antenna : 0
Short : 0
Overlap : 0
End Summary
Verification Complete : 0 Viols. 0 Wrngs.
**********End: VERIFY GEOMETRY**********
*** verify geometry (CPU: 0:00:00.2 MEM: 168.1M)

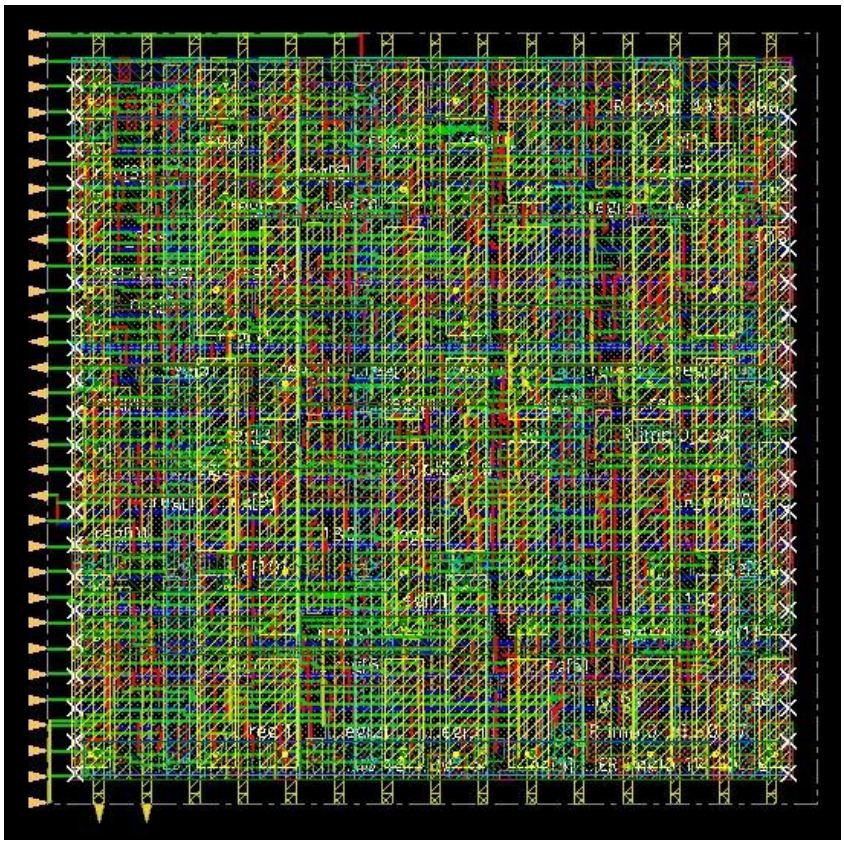


ADD FILLERS



ADD METAL FILL

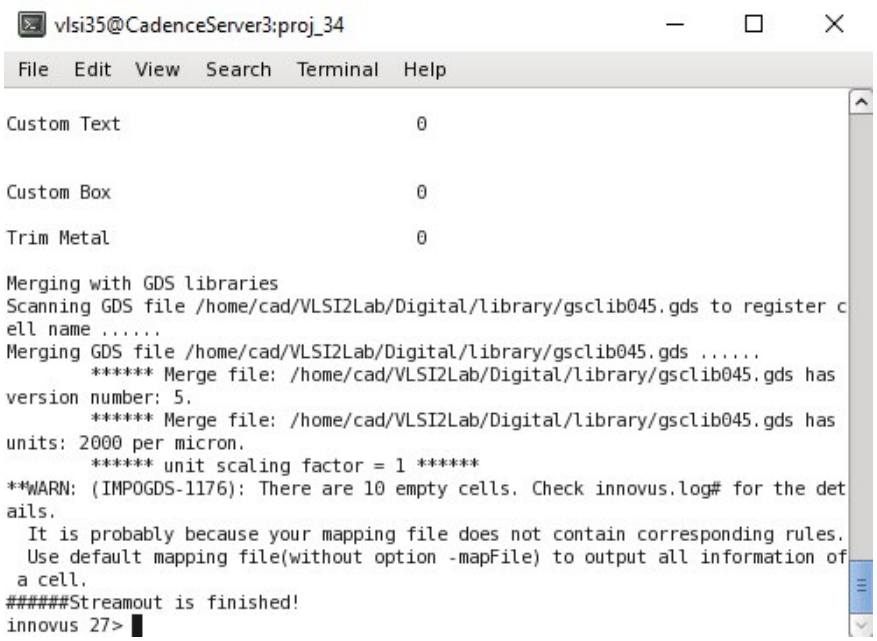




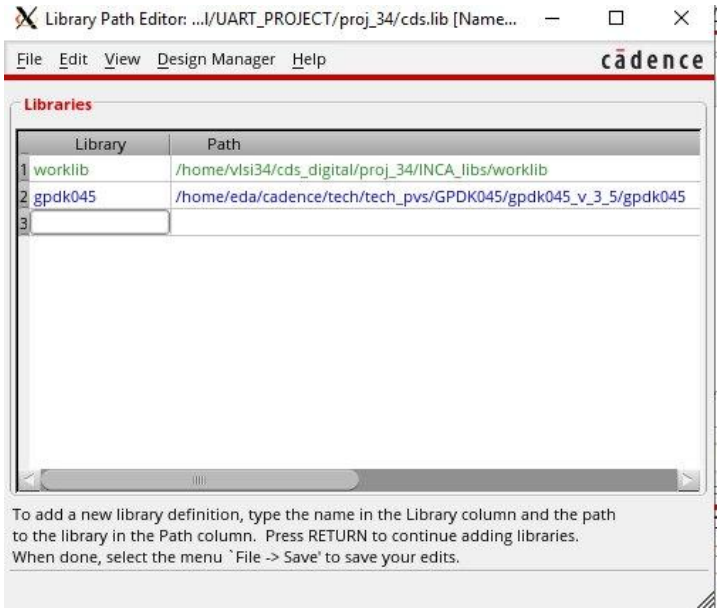
DRC AND LVS VERIFICATION

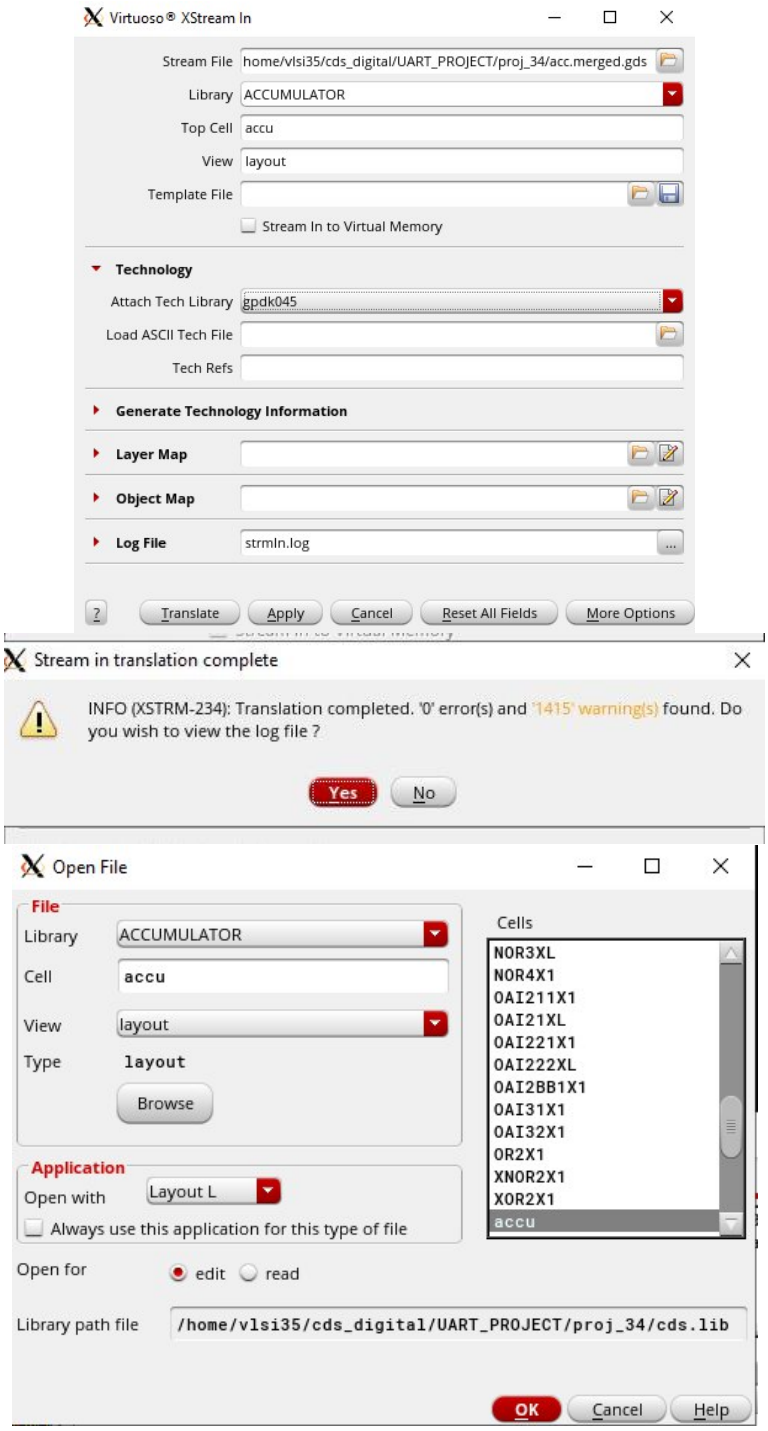
The design is exported to GDS format and LVS netlist for verification in Virtuoso. Design Rule Check (DRC) ensures there are no layout violations, and Layout Versus Schematic (LVS) confirms that the layout matches the synthesized netlist.

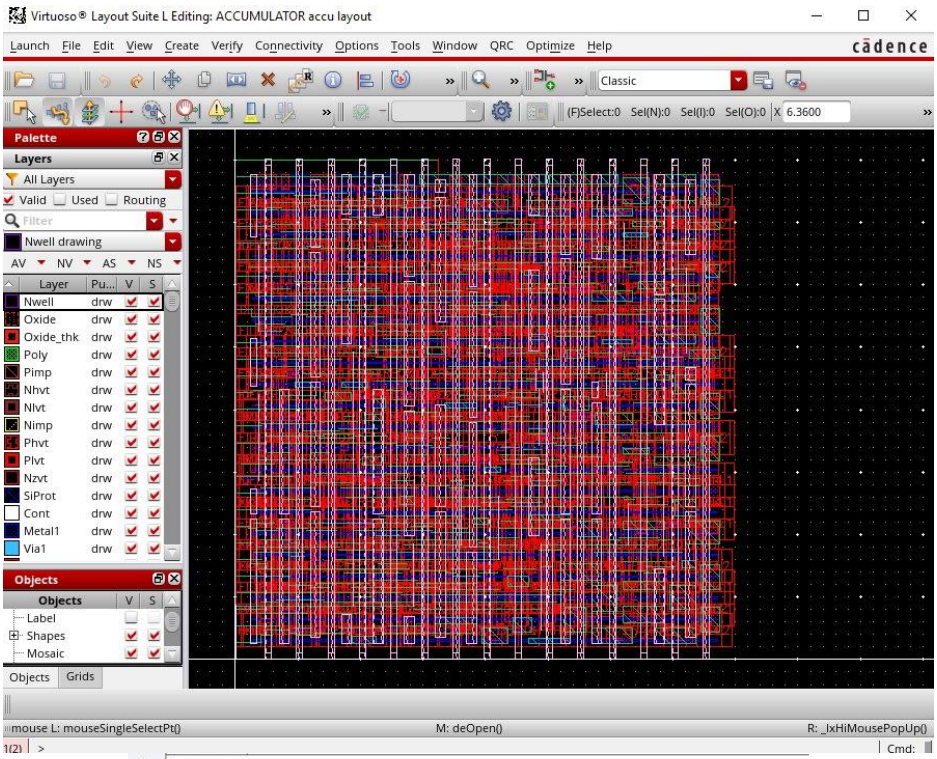
Exporting Data
to export GDS & netlist to run DRC and LVS in Virtuoso. The command source /home/cad/VLSI2Lab/Digital/PnR/export_gds.tcl this will generate two files accu.lvs_netlist.vg & acc.merged.gds in the run directory.

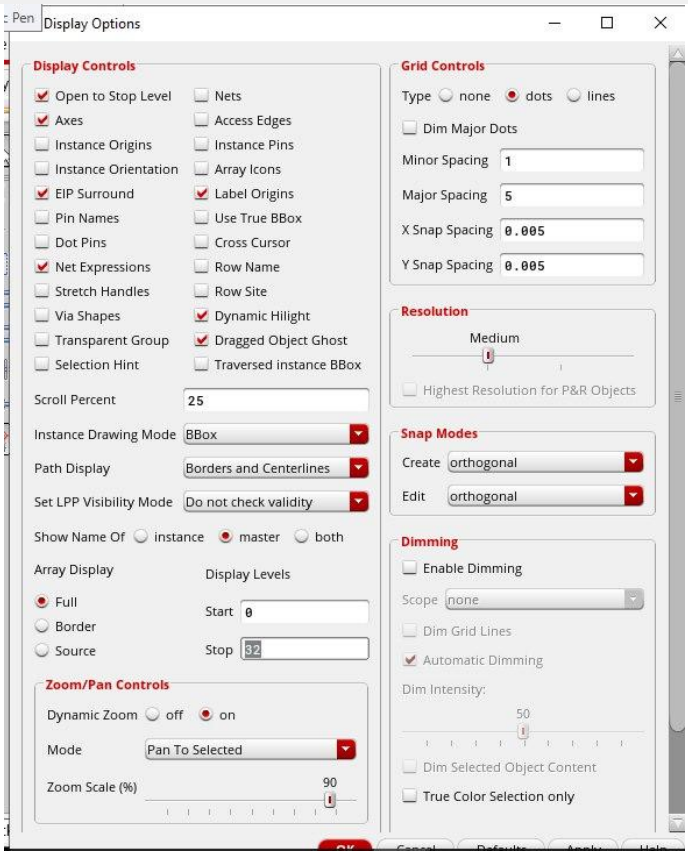

vlsi35@CadenceServer3:proj_34
File Edit View Search Terminal Help
Custom Text 0
Custom Box 0
Trim Metal 0
Merging with GDS libraries
Scanning GDS file /home/cad/VLSI2Lab/Digital/library/gsclib045.gds to register cell name ......
Merging GDS file /home/cad/VLSI2Lab/Digital/library/gsclib045.gds ......
****** Merge file: /home/cad/VLSI2Lab/Digital/library/gsclib045.gds has version number: 5.
****** Merge file: /home/cad/VLSI2Lab/Digital/library/gsclib045.gds has units: 2000 per micron.
****** unit scaling factor = 1 ******
**WARN: (IMPOGDS-1176): There are 10 empty cells. Check innovus.log# for the details.
It is probably because your mapping file does not contain corresponding rules.
Use default mapping file(without option -mapFile) to output all information of a cell.
######Streamout is finished!
innovus 27>

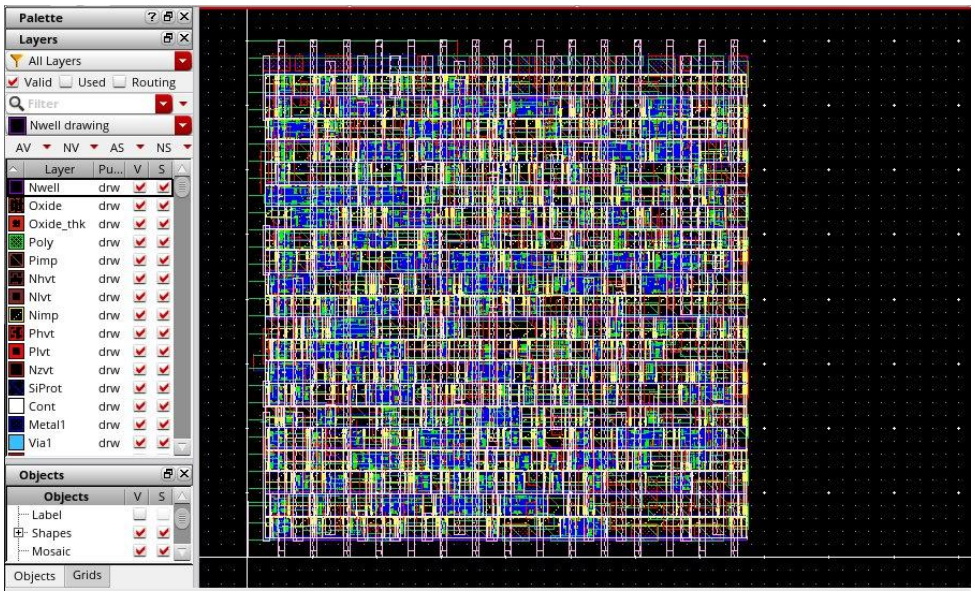
ADD LIBRARY


Library Path Editor: ...l/UART_PROJECT/proj_34/cds.lib [Name...
File Edit View Design Manager Help
cadence
Libraries
Library Path
1 worklib /home/vlsi34/cds_digital/proj_34/INCA_libs/worklib
2 gpdk045 /home/eda/cadence/tech/tech_pvs/GPDK045/gpdk045_v_3_5/gpdk045
3
To add a new library definition, type the name in the Library column and the path to the library in the Path column. Press RETURN to continue adding libraries.
When done, select the menu `File -> Save' to save your edits.


Virtuoso® XStream In
Stream File
home/vlsi35/cds_digital/UART_PROJECT/proj_34/acc.merged.gds
Library
ACCUMULATOR
Top Cell
accu
View
layout
Template File
Stream In to Virtual Memory
Technology
Attach Tech Library
gpdk045
Load ASCII Tech File
Tech Refs
Generate Technology Information
Layer Map
Object Map
Log File
strmIn.log
Translate
Apply
Cancel
Reset All Fields
More Options
Stream in translation complete
INFO (XSTRM-234): Translation completed. '0' error(s) and '1415' warning(s) found. Do you wish to view the log file ?
Yes
No
Open File
File
Library
ACCUMULATOR
Cell
accu
View
layout
Type
layout
Browse
Application
Open with
Layout L
Always use this application for this type of file
Cells
NOR3XL
NOR4X1
OAI211X1
OAI21XL
OAI221X1
OAI222XL
OAI2BB1X1
OAI31X1
OAI32X1
OR2X1
XNOR2X1
XOR2X1
accu
Open for
edit
read
Library path file
/home/vlsi35/cds_digital/UART_PROJECT/proj_34/cds.lib
OK
Cancel
Help

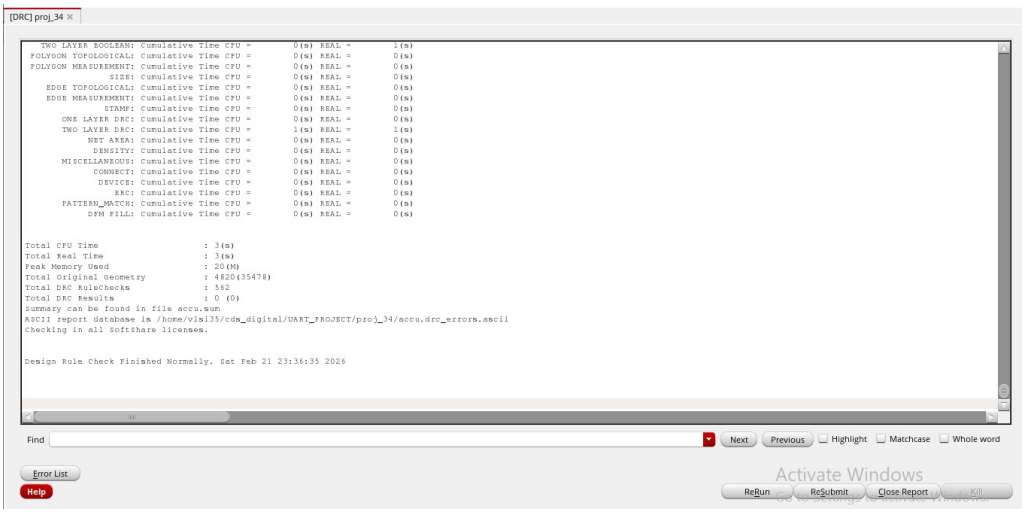

Virtuoso® Layout Suite L Editing: ACCUMULATOR accu layout
Launch File Edit View Create Verify Connectivity Options Tools Window QRC Optimize Help
cadence
Palette
Layers
All Layers
Valid Used Routing
Nwell drawing
Layer Pu... V S
Nwell drw
Oxide drw
Oxide_thk drw
Poly drw
Pimp drw
Nhvt drw
Nlvt drw
Nimp drw
Phvt drw
Plvt drw
Nzvt drw
SiProt drw
Cont drw
Metal1 drw
Via1 drw
Objects
Label
Shapes
Mosaic
Objects Grids
mouse L: mouseSingleSelectPt()
M: deOpen()
R: _lxHiMousePopUp()
Cmd:

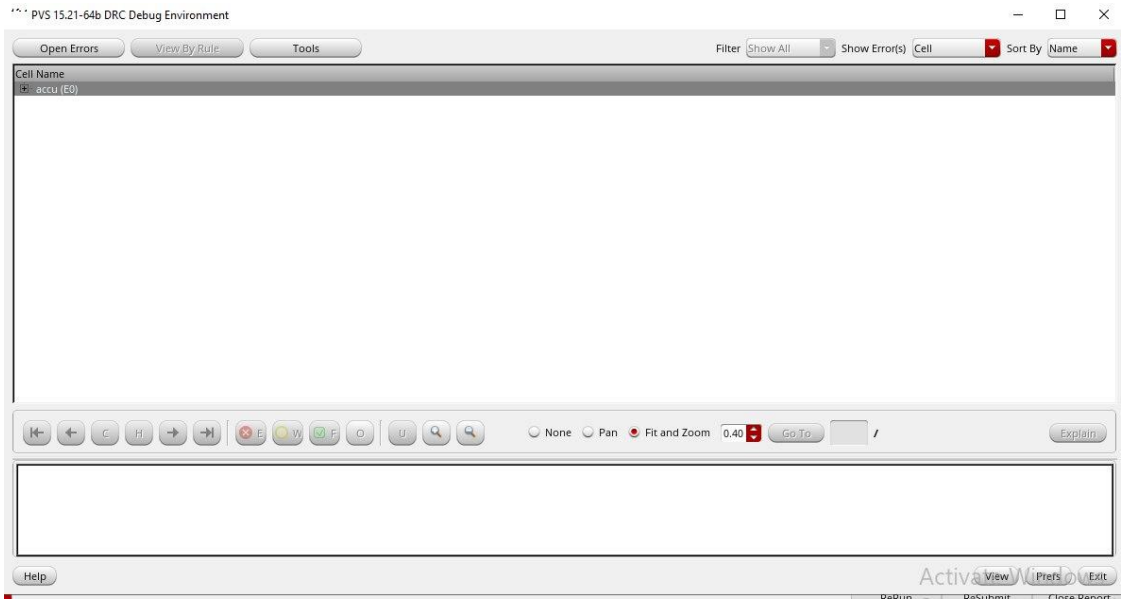

Display Options
Display Controls
Open to Stop Level
Axes
Instance Origins
Instance Orientation
EIP Surround
Pin Names
Dot Pins
Net Expressions
Stretch Handles
Via Shapes
Transparent Group
Selection Hint
Nets
Access Edges
Instance Pins
Array Icons
Label Origins
Use True BBox
Cross Cursor
Row Name
Row Site
Dynamic Hilight
Dragged Object Ghost
Traversed instance BBox
Scroll Percent 25
Instance Drawing Mode BBox
Path Display Borders and Centerlines
Set LPP Visibility Mode Do not check validity
Show Name Of instance master both
Array Display
Full
Border
Source
Display Levels
Start 0
Stop 32
Zoom/Pan Controls
Dynamic Zoom off on
Mode Pan To Selected
Zoom Scale (%) 90
Grid Controls
Type none dots lines
Dim Major Dots
Minor Spacing 1
Major Spacing 5
X Snap Spacing 0.005
Y Snap Spacing 0.005
Resolution
Medium
Highest Resolution for P&R Objects
Snap Modes
Create orthogonal
Edit orthogonal
Dimming
Enable Dimming
Scope none
Dim Grid Lines
Automatic Dimming
Dim Intensity:
50
Dim Selected Object Content
True Color Selection only



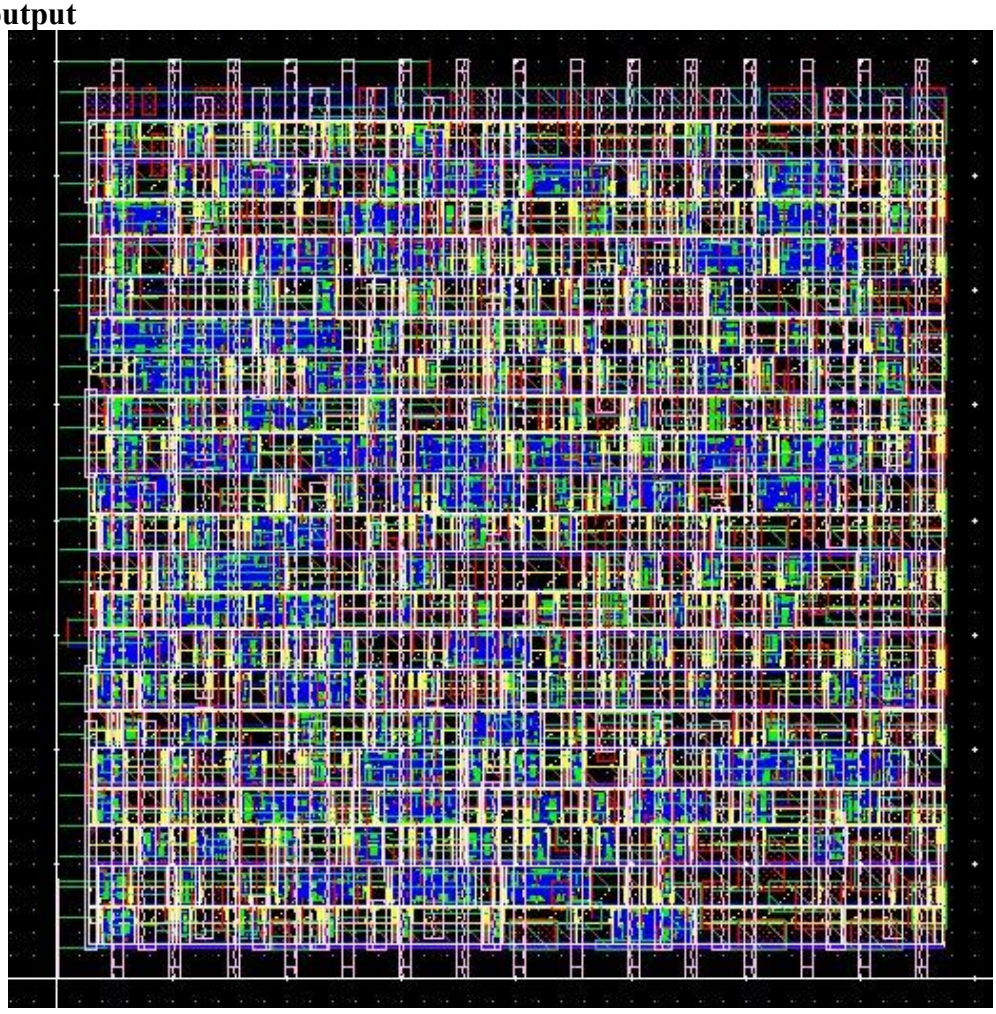
FINAL OUTPUT

The final layout is generated in Innovus after successful timing closure, DRC, and LVS verification. The design achieves zero violations and is ready for fabrication-level validation.

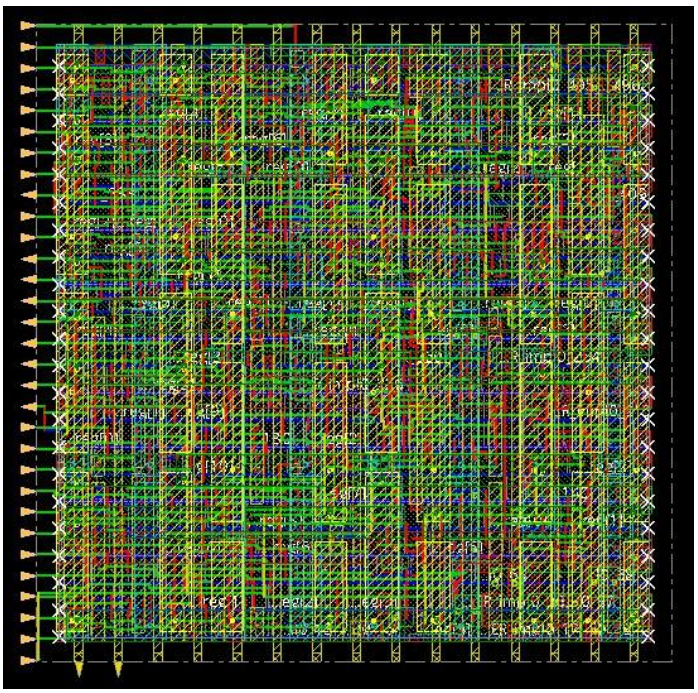




Final output



Final Layout Innovus



```
------------------------------------------------------------
     optDesign Final SI Timing Summary
------------------------------------------------------------

Setup views included:
 func@BC_rcbest0.hold
Hold  views included:
 func@BC_rcbest0.hold

+--------------------+--------+--------+--------+
|     Setup mode     |   all  | reg2reg | default |
+--------------------+--------+--------+--------+
|           WNS (ns):|  2.023 |  2.023 |  2.191 |
|           TNS (ns):|  0.000 |  0.000 |  0.000 |
|    Violating Paths:|    0   |    0   |    0   |
|          All Paths:|   92   |   46   |   70   |
+--------------------+--------+--------+--------+

+--------------------+--------+--------+--------+
|     Hold mode      |   all  | reg2reg | default |
+--------------------+--------+--------+--------+
|           WNS (ns):|  0.027 |  0.079 |  0.027 |
|           TNS (ns):|  0.000 |  0.000 |  0.000 |
|    Violating Paths:|    0   |    0   |    0   |
|          All Paths:|   92   |   46   |   70   |
+--------------------+--------+--------+--------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout   |      0 (0)       |     0      |      0 (0)       |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 42.063%
Total number of glitch violations: 0
------------------------------------------------------------
**optDesign ... cpu = 0:00:09, real = 0:00:11, mem = 1521.5M, totSessionCpu=0:02
:20 **
 ReSet Options after AAE Based Opt flow
*** Finished optDesign ***
Removing temporary dont_use automatically set for cells with technology sites wi
th no row.
0
```

```
Stream Out Information Processed for GDS version 5:
Units: 2000 DBU

Object                                             Count
----------------------------------------------------------
Instances                                            706

Ports/Pins                                            59
     metal layer Metal3                               30
     metal layer Metal4                               29

Nets                                                2110
     metal layer Metal1                              239
     metal layer Metal2                             1128
     metal layer Metal3                              680
     metal layer Metal4                               63

     Via Instances                                 1394

Special Nets                                          35
     metal layer Metal1                               21
     metal layer Metal4                               14

     Via Instances                                  495

Metal Fills                                         190
     metal layer Metal1                                3
     metal layer Metal2                               75
     metal layer Metal3                               67
     metal layer Metal4                               45

     Via Instances                                  406

Metal FillOPCs                                        0

     Via Instances                                    0

Text                                                298
     metal layer Metal1                               52
     metal layer Metal2                              119
     metal layer Metal3                               94
     metal layer Metal4                               33


Blockages                                             0


Custom Text                                           0
```

```
------------------------------------------------------------
     optDesign Final SI Timing Summary
------------------------------------------------------------

Setup views included:
 func@BC_rcbest0.hold
Hold  views included:
 func@BC_rcbest0.hold

+--------------------+--------+--------+--------+
|     Setup mode     |   all  | reg2reg | default |
+--------------------+--------+--------+--------+
|           WNS (ns):|  2.023 |  2.023 |  2.191 |
|           TNS (ns):|  0.000 |  0.000 |  0.000 |
|    Violating Paths:|    0   |    0   |    0   |
|          All Paths:|   92   |   46   |   70   |
+--------------------+--------+--------+--------+

+--------------------+--------+--------+--------+
|     Hold mode      |   all  | reg2reg | default |
+--------------------+--------+--------+--------+
|           WNS (ns):|  0.027 |  0.079 |  0.027 |
|           TNS (ns):|  0.000 |  0.000 |  0.000 |
|    Violating Paths:|    0   |    0   |    0   |
|          All Paths:|   92   |   46   |   70   |
+--------------------+--------+--------+--------+

+---------------+-----------------------------+------------------+
|               |             Real            |       Total      |
|    DRVs       +-----------------------------+------------------|
|               |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+---------------+------------------+------------+------------------+
|   max_cap     |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran    |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout  |      0 (0)       |     0      |      0 (0)       |
|   max_length  |      0 (0)       |     0      |      0 (0)       |
+---------------+------------------+------------+------------------+

Density: 42.063%
Total number of glitch violations: 0
------------------------------------------------------------
**optDesign ... cpu = 0:00:09, real = 0:00:11, mem = 1521.5M, totSessionCpu=0:02
:20 **
 ReSet Options after AAE Based Opt flow
*** Finished optDesign ***
Removing temporary dont_use automatically set for cells with technology sites wi
th no row.
0
```

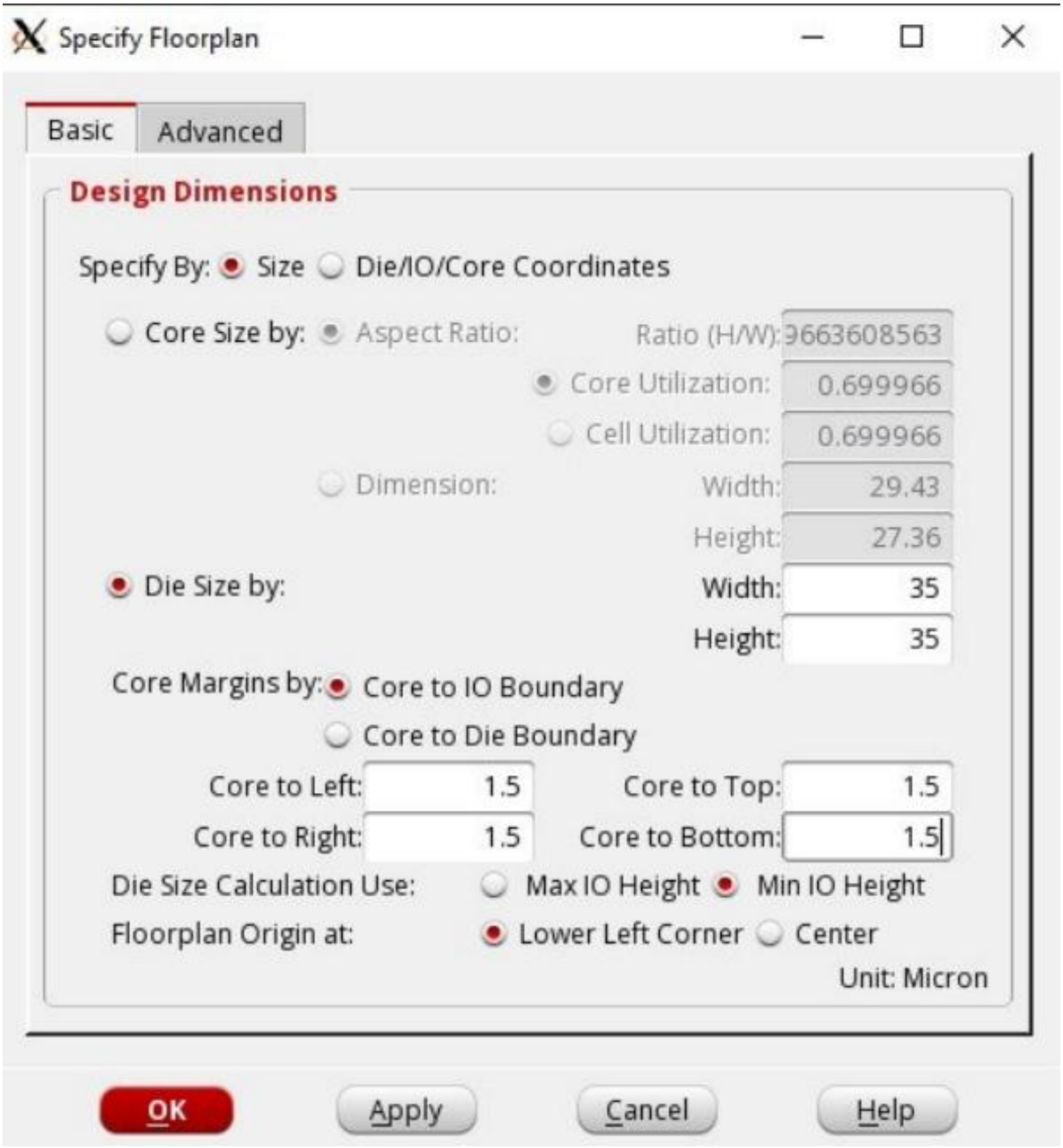
EXTRACT RC

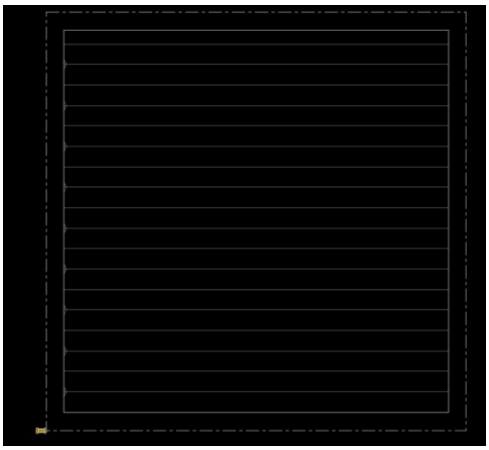
```
Extraction of Geometries DONE (NETS: 239  CPU Time: 0:00:00.4  Real Time: 0:00:0
1.0  MEM: 1590.559M)

Parasitic Network Creation STARTED
.....................
Number of Extracted Resistors     : 3023
Number of Extracted Ground Caps   : 3268
Number of Extracted Coupling Caps : 4
Parasitic Network Creation DONE (Nets: 239  CPU Time: 0:00:00.2  Real Time: 0:00
:00.0  MEM: 1626.566M)

IQRC Extraction engine is being closed...
IQRC Fullchip Extraction DONE (CPU Time: 0:00:02.4  Real Time: 0:00:02.0  MEM: 1
626.562M)
0
innovus 29>
```

OPTIMIZATION

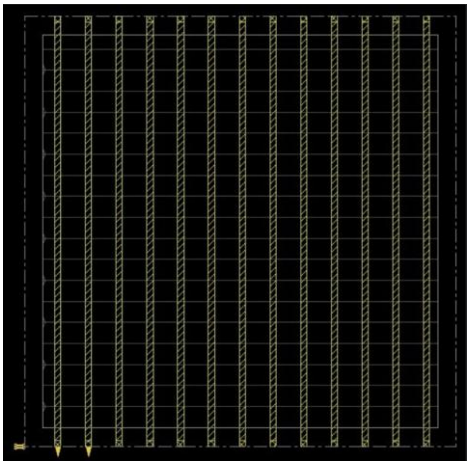
I. Die size of 35x35 with pin spacing of 0.8

DIE SIZE

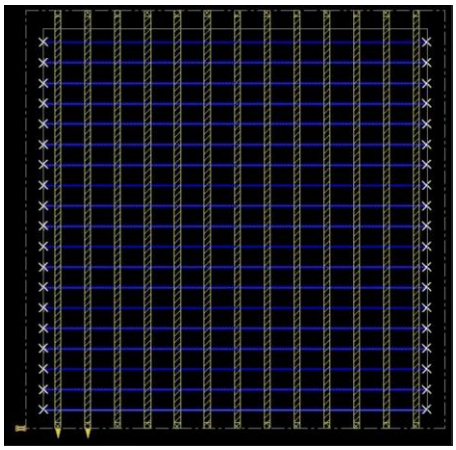




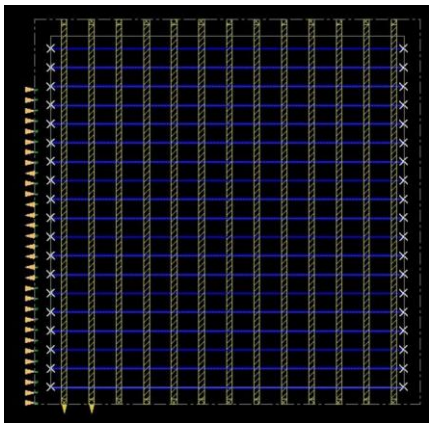
POWER PLANNING (POWER MESH)



SROUTE

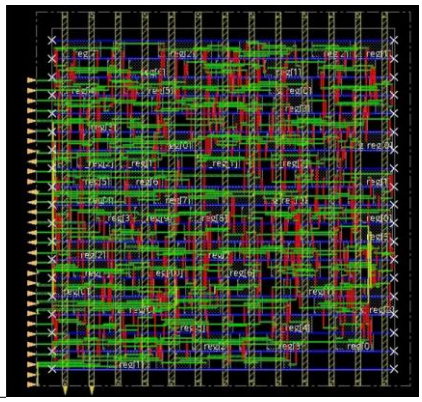


PIN ASSIGNMENT (PIN EDITOR)



Place_opt_design

```
------------------------------------------------------------
     optDesign Final Summary
------------------------------------------------------------

Setup views included:
 funcBC_rcbest0.hold

+--------------------+--------+--------+--------+
|     Setup mode     |  all   | reg2reg | default |
+--------------------+--------+--------+--------+
|           WNS (ns):| 2.035  | 2.035  | 2.206  |
|           TNS (ns):| 0.000  | 0.000  | 0.000  |
|    Violating Paths:|   0    |   0    |   0    |
|         All Paths: |   92   |   46   |   70   |
+--------------------+--------+--------+--------+

+----------------+-----------------------------------+------------------+
|                |              Real                 |       Total      |
|     DRVs       +------------------+----------------+------------------+
|                |  Nr nets(terms)  |  Worst Vio     |  Nr nets(terms)  |
+----------------+------------------+----------------+------------------+
|   max_cap      |      0 (0)       |     0.000      |      0 (0)       |
|   max_tran     |      0 (0)       |     0.000      |      0 (0)       |
|   max_fanout   |      0 (0)       |        0       |      0 (0)       |
|   max_length   |      0 (0)       |        0       |      0 (0)       |
+----------------+------------------+----------------+------------------+

Density: 56.694%
Routing Overflow: 0.00% H and 0.00% V
------------------------------------------------------------
**optDesign ... cpu = 0:00:07, real = 0:00:07, mem = 1348.6M, totSessionCpu=0:01
:59 **
**WARN: (IMPOPT-3195):  Analysis mode has changed.
Type 'man IMPOPT-3195' for more detail.
*** Finished optDesign ***
Removing temporary dont_use automatically set for cells with technology sites wi
th no row.
*** Free Virtual Timing Model ...(mem=1348.6M)
**place_opt_design ... cpu = 0:00:11, real = 0:00:13, mem = 1257.8M **
*** Finished GigaPlace ***
```

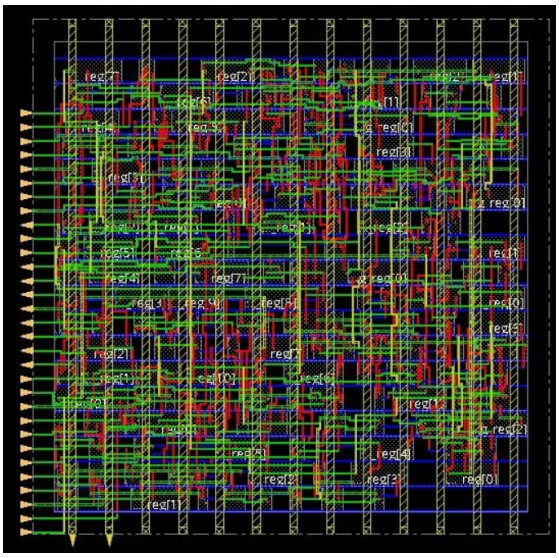


Report Timing

```
innovus 15> report_timing
#################################################################
#  Generated by:      Cadence Innovus 16.10-p004_1
#  OS:                Linux x86_64(Host ID CadenceServer3.localdomain)
#  Generated on:      Tue Mar 24 22:57:10 2026
#  Design:            uart
#  Command:           report_timing
#################################################################
Path 1: MET Setup Check with Pin uart_rx_unit/n_reg_reg[1]/CK
Endpoint:   uart_rx_unit/n_reg_reg[1]/D (^) checked with  leading edge of 'func_
clk'
Beginpoint: bdgen/r_reg_reg[6]/Q        (v) triggered by  leading edge of 'func_
clk'
Path Groups: {func_clk}
Analysis View: funcBC_rcbest0.hold
Other End Arrival Time          0.000
- Setup                         0.026
+ Phase Shift                   2.500
= Required Time                 2.474
- Arrival Time                  0.438
= Slack Time                    2.036
     Clock Rise Edge                     0.000
     + Clock Network Latency (Prop)      0.000
     = Beginpoint Arrival Time           0.000
     +-------------------------------------------------------------------------+
     |          Instance          |     Arc     |   Cell   | Delay | Arrival | Required |
     |                            |             |          |       |  Time   |   Time   |
     |----------------------------+-------------+----------+-------+---------+----------|
     | bdgen/r_reg_reg[6]         | CK ^        |          |       |   0.000 |    2.036 |
     | bdgen/r_reg_reg[6]         | CK ^ -> Q v | DFFRHQX1 | 0.061 |   0.061 |    2.097 |
     | bdgen/g176                 | B v -> Y ^  | NOR4X1   | 0.039 |   0.100 |    2.136 |
     | bdgen/g174                 | C ^ -> Y ^  | AND4X1   | 0.076 |   0.176 |    2.212 |
     | uart_rx_unit/g911          | AN ^ -> Y ^ | NOR2BX1  | 0.031 |   0.207 |    2.243 |
     | uart_rx_unit/g1710         | B ^ -> Y v  | NAND2X1  | 0.078 |   0.285 |    2.321 |
     | uart_rx_unit/g1708         | A v -> Y ^  | INVX1    | 0.044 |   0.329 |    2.365 |
     | uart_rx_unit/g1681         | C ^ -> Y v  | NAND3XL  | 0.037 |   0.366 |    2.402 |
     | uart_rx_unit/g1668         | C v -> Y ^  | NAND3XL  | 0.030 |   0.396 |    2.432 |
     | uart_rx_unit/g1662         | B ^ -> Y v  | NAND2XL  | 0.026 |   0.422 |    2.458 |
     | uart_rx_unit/g1660         | B0 v -> Y ^ | OAI21XL  | 0.016 |   0.438 |    2.474 |
     | uart_rx_unit/n_reg_reg[1]  | D ^         | DFFRHQX1 | 0.000 |   0.438 |    2.474 |
     +-------------------------------------------------------------------------+
```

```
innovus 16> report_timing -early
#################################################################
#  Generated by:      Cadence Innovus 16.10-p004_1
#  OS:                Linux x86_64(Host ID CadenceServer3.localdomain)
#  Generated on:      Tue Mar 24 22:59:22 2026
#  Design:            uart
#  Command:           report_timing -early
#################################################################
#################################################################################
# Design Stage: PreRoute
# Design Name: uart
# Design Mode: 90nm
# Analysis Mode: MMMC Non-OCV
# Parasitics Mode: No SPEF/RCDB
# Signoff Settings: SI Off
#################################################################################
Calculate delays in Single mode...
Total number of fetched objects 239
AAE_INFO: Total number of nets for which stage creation was skipped for all views 0
End delay calculation. (MEM=1402.56 CPU=0:00:00.0 REAL=0:00:00.0)
Path 1: MET Removal Check with Pin uart_tx_unit/state_reg_reg[1]/CK
Endpoint:   uart_tx_unit/state_reg_reg[1]/RN (^) checked with  leading edge of
'func_clk'
Beginpoint: reset                       (v) triggered by  leading edge of
'g'
Path Groups: {async_default}
Analysis View: funcBC_rcbest0.hold
Other End Arrival Time          0.000
+ Removal                       0.044
+ Phase Shift                   0.000
= Required Time                 0.045
  Arrival Time                  0.075
  Slack Time                    0.030
    Clock Rise Edge                     0.000
    + Input Delay                       0.000
    = Beginpoint Arrival Time           0.000
    +-------------------------------------------------------------------+
    |          Instance          |  Arc  |  Cell  | Delay | Arrival |  Required |
    |                            |       |        |       |  Time   |    Time   |
    |----------------------------+-------+--------+-------+---------+-----------|
    |                            | reset v |      |       |   0.000 |    -0.030 |
    | uart_tx_unit/g1059         | A v -> Y ^ | INVX1 | 0.075 |   0.075 |    0.044 |
    | uart_tx_unit/state_reg_reg[1] | RN ^ | DFFRHQX1 | 0.000 |   0.075 |    0.045 |
    +-------------------------------------------------------------------+
```

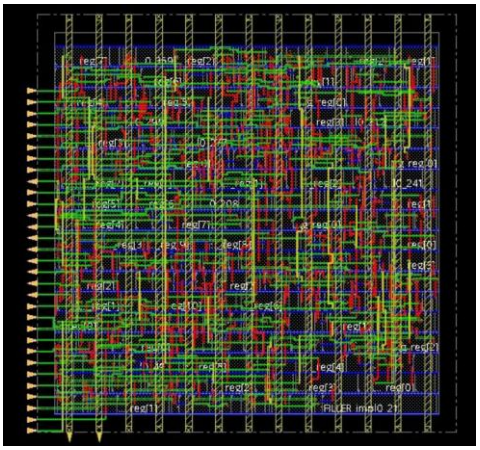
Nanoroute



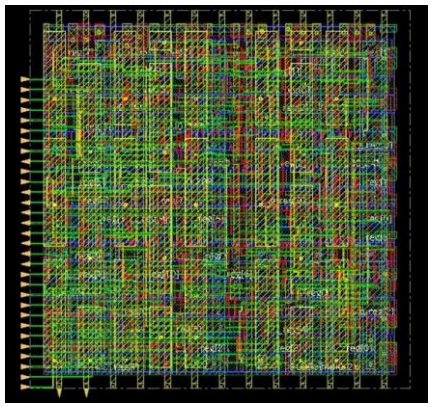
Post-route

```
------------------------------------------------------------
          timeDesign Summary
------------------------------------------------------------

Hold  views included:
 func@BC_rcbest0.hold

+--------------------+---------+---------+---------+
|     Hold mode      |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  0.033  |  0.080  |  0.033  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|   92    |   46    |   70    |
+--------------------+---------+---------+---------+

Density: 56.694%
------------------------------------------------------------
Reported timing to dir ./timingReports
Total CPU time: 0.61 sec
Total Real time: 1.0 sec
Total Memory Usage: 1374.519531 Mbytes
Reset AAE Options
innovus 22> innovus 22>
```

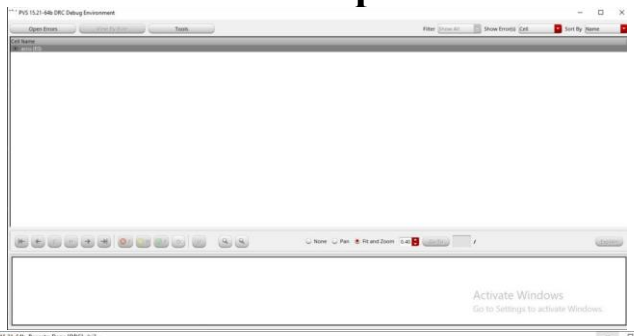
Add Fillers

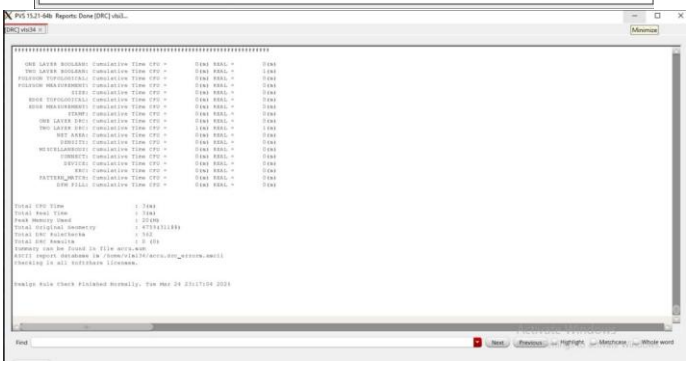


Metalfill

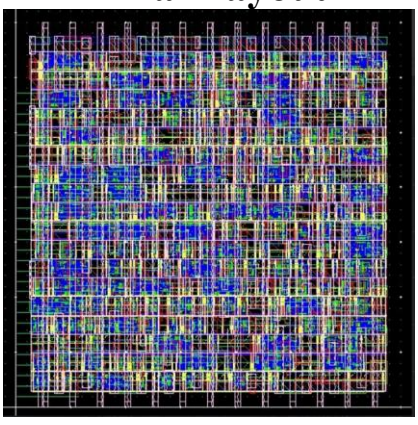


DRC Report

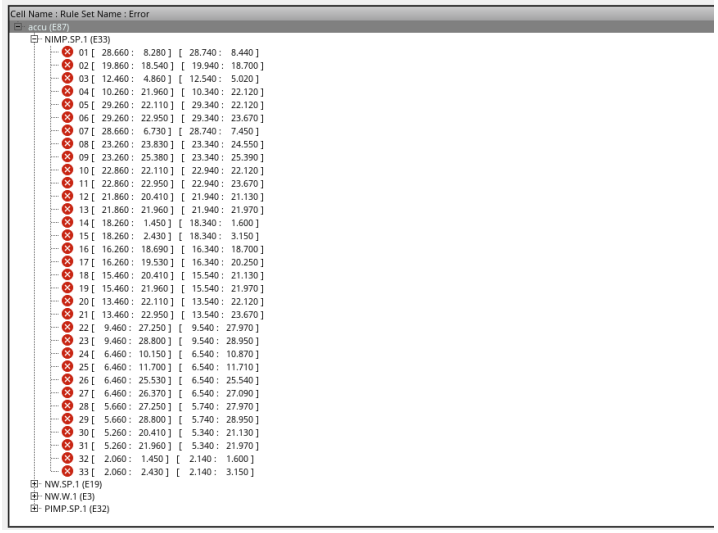




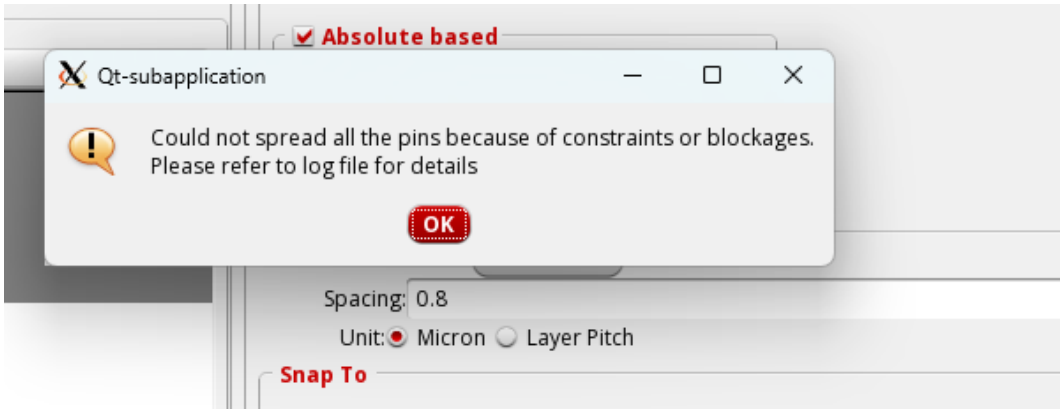
Final Layout



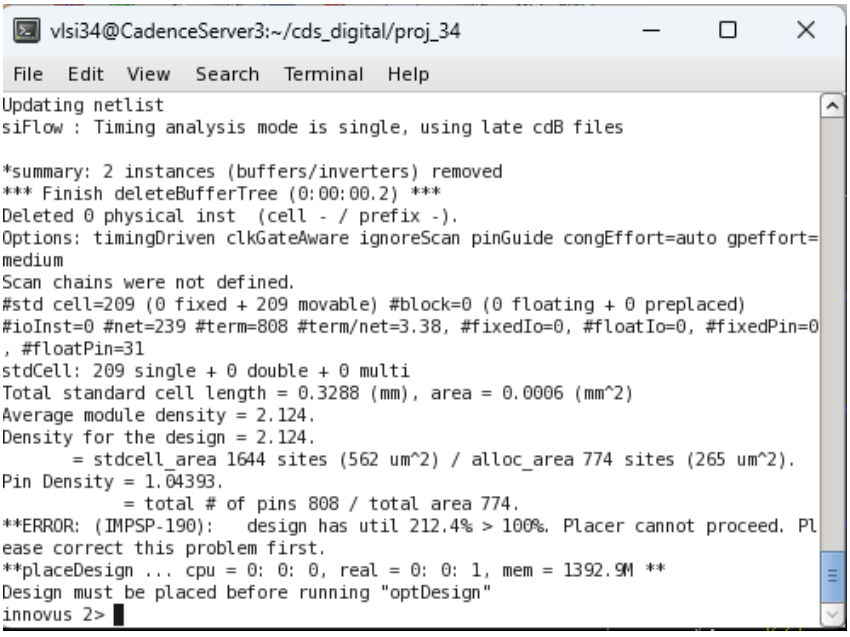
II. For Die size of 32x32 with pin spacing of 0.8 and 30x30 with pin spacing of 0.8 the DRC was found with errors.



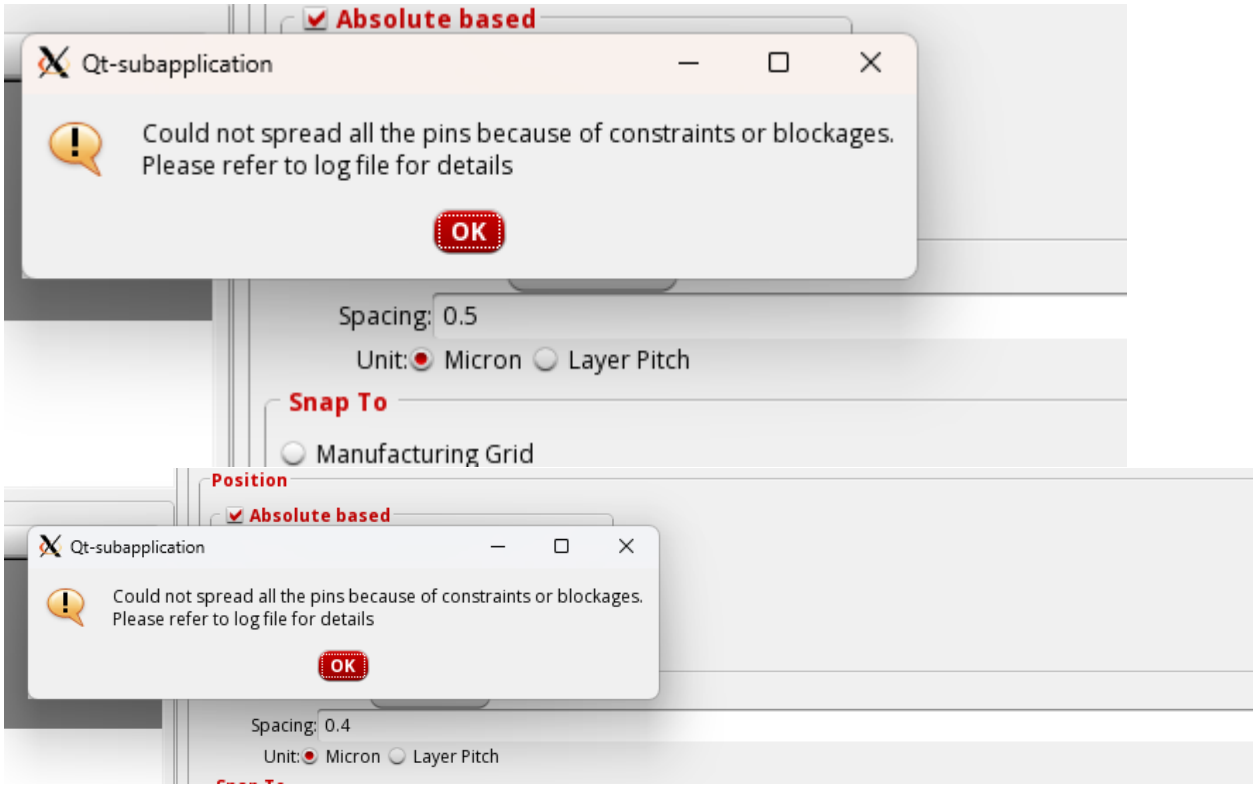
III. For Die size of 26x26 with pin spacing of 0.8 there was error with pin editor.

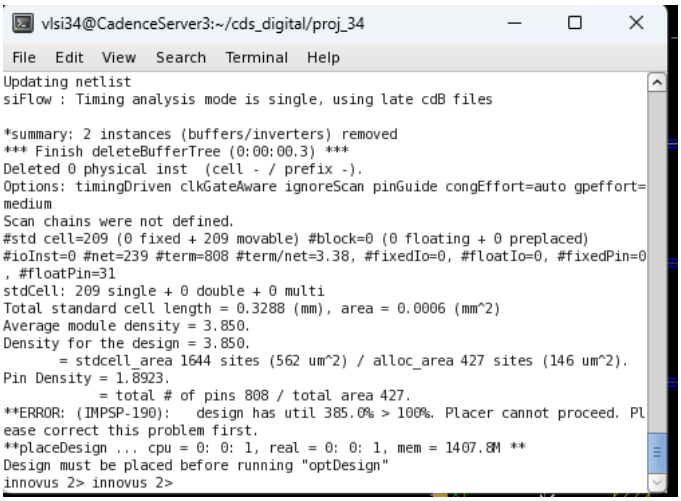


IV. For Die size of 20x20 with pin spacing of 0.4 there was error with high utilization.

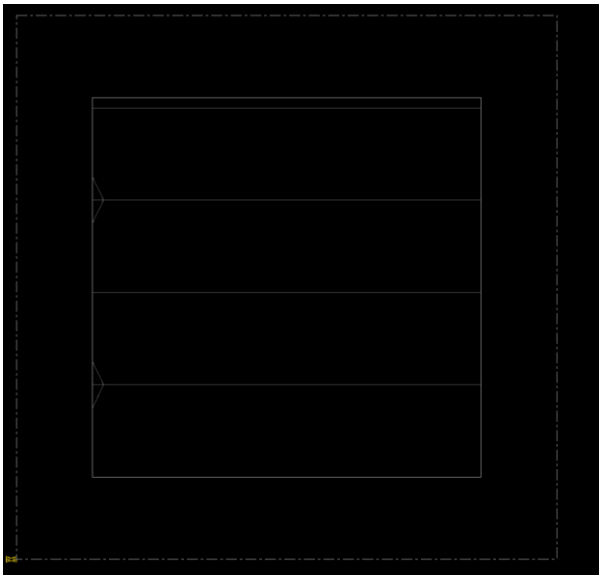


V. For Die size of 15x15 there was error with pin editor for spacing of 0.5,0.4 and optimal design step for 0.8 due to high utilization.

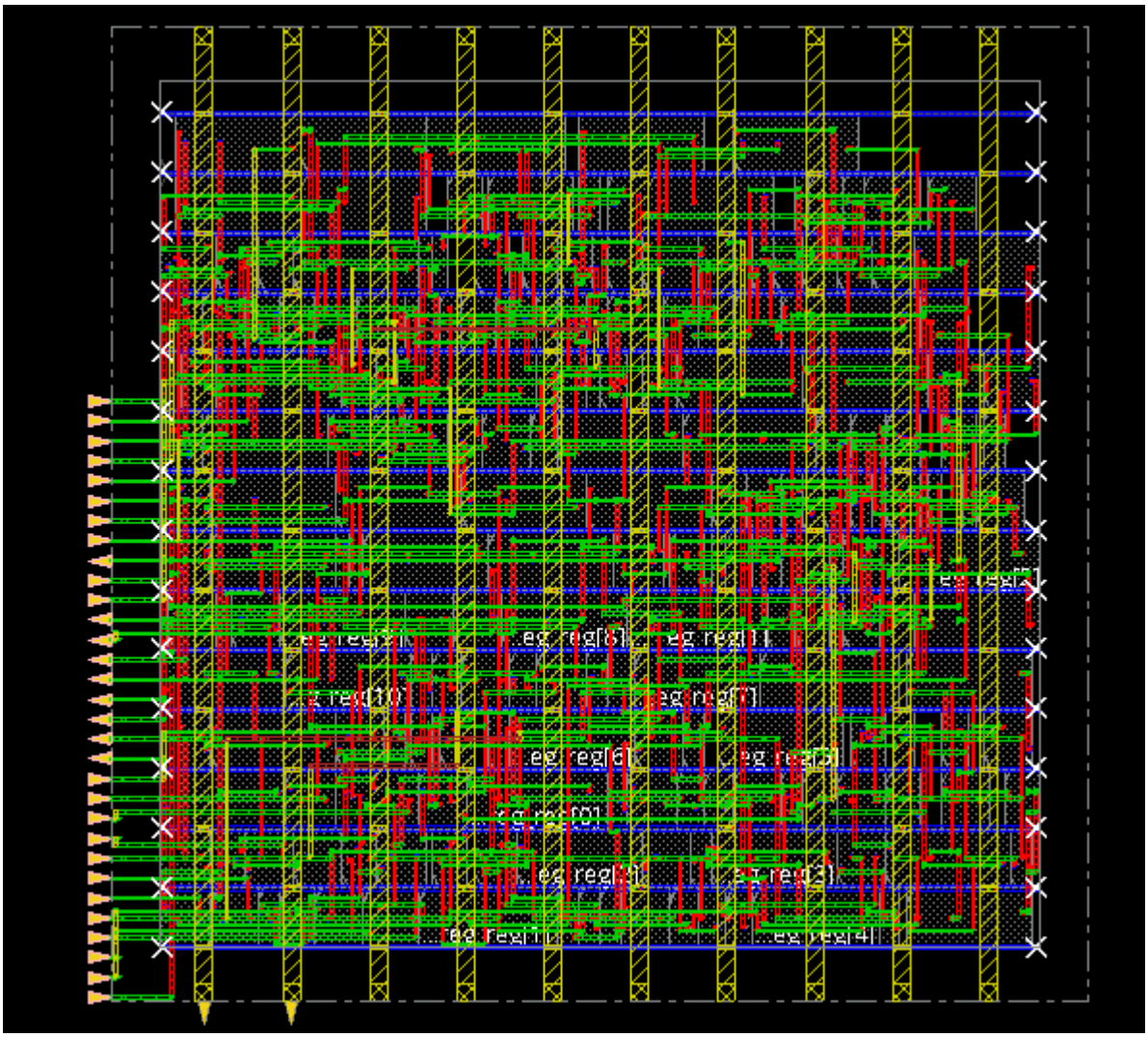




VI. For Die size of 6.5x6.5 and 10x10 there was uneven spacing seen.

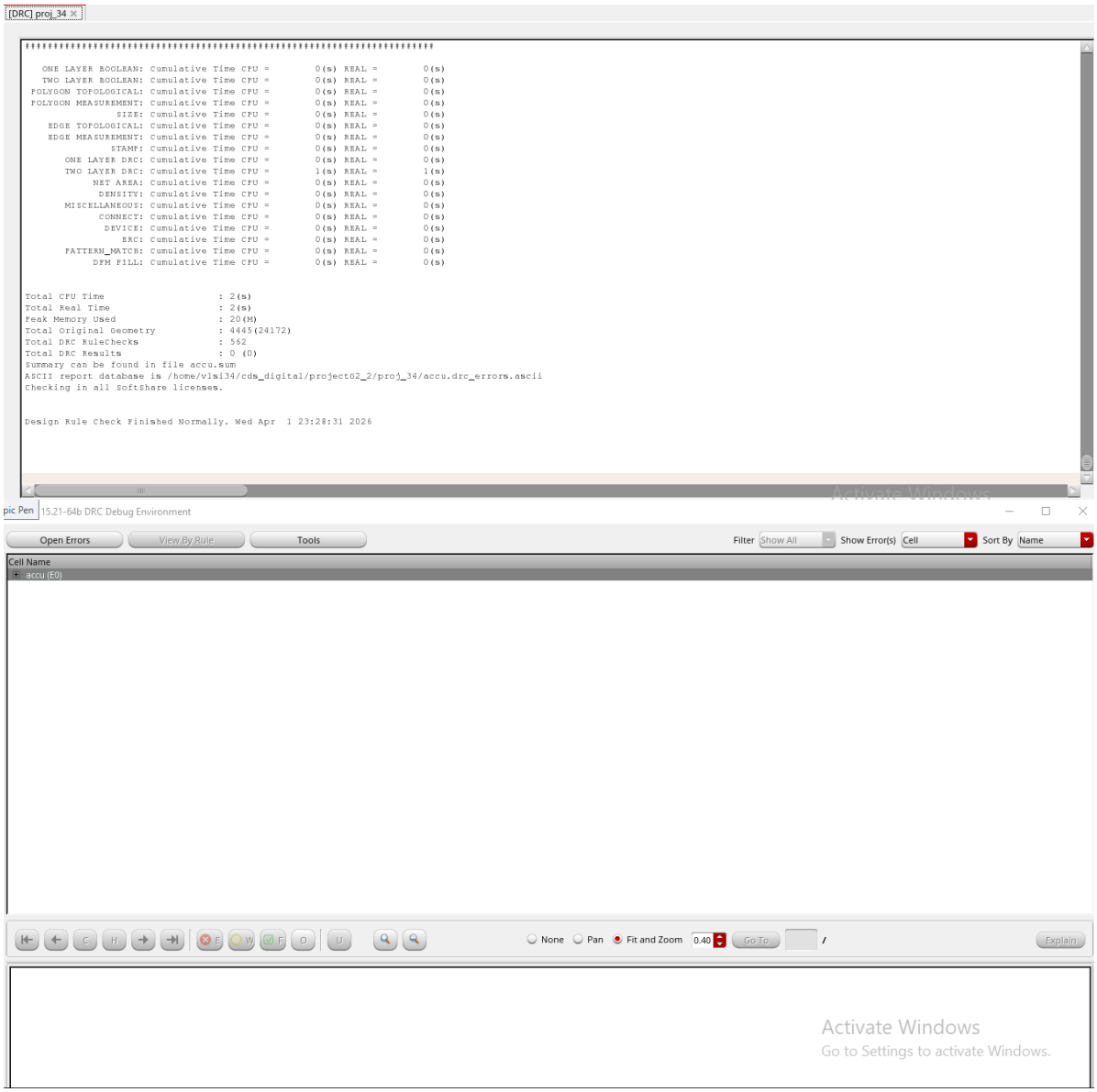


VII. After several trial and error, the final optimized die size was found to be 28x28, maintaining 0.4microns of spacing between the (31) pins



```
------------------------------------------------------------
     optDesign Final Summary
------------------------------------------------------------

Setup views included:
 func@BC_rcbest0.hold

+--------------------+---------+---------+---------+
|     Setup mode     |   all   | reg2reg | default |
+--------------------+---------+---------+---------+
|           WNS (ns):|  2.042  |  2.042  |  2.211  |
|           TNS (ns):|  0.000  |  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |    0    |
|          All Paths:|   92    |   46    |   70    |
+--------------------+---------+---------+---------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout   |      0 (0)       |     0      |      0 (0)       |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 93.141%
Routing Overflow: 0.00% H and 0.00% V
```

```
Path 1: MET Setup Check with Pin uart_rx_unit/n_reg_reg[1]/CK
Endpoint:   uart_rx_unit/n_reg_reg[1]/D (^) checked with  leading edge of 'func_
clk'
Beginpoint: bdgen/r_reg_reg[7]/Q       (v) triggered by  leading edge of 'func_
clk'
Path Groups: {func_clk}
Analysis View: func@BC_rcbest0.hold
Other End Arrival Time          0.000
- Setup                         0.026
+ Phase Shift                   2.500
= Required Time                 2.474
- Arrival Time                  0.433
= Slack Time                    2.040
     Clock Rise Edge                 0.000
     + Clock Network Latency (Prop)  0.000
     = Beginpoint Arrival Time       0.000
     +-------------------------------------------------------------------------------+
     |         Instance          |     Arc     |   Cell    | Delay | Arrival | Required |
     |                           |             |           |       |  Time   |   Time   |
     |---------------------------+-------------+-----------+-------+---------+----------|
     | bdgen/r_reg_reg[7]        | CK ^        |           |       |   0.000 |    2.041 |
     | bdgen/r_reg_reg[7]        | CK ^ -> Q v | DFFRHQX1  | 0.060 |   0.061 |    2.101 |
     | bdgen/g176                | A v -> Y ^  | NOR4X1    | 0.042 |   0.102 |    2.143 |
     | bdgen/g174                | C ^ -> Y ^  | AND4X1    | 0.077 |   0.179 |    2.220 |
     | uart_rx_unit/g911         | AN ^ -> Y ^ | NOR2BX1   | 0.031 |   0.210 |    2.251 |
     | uart_rx_unit/g1710        | B ^ -> Y v  | NAND2X1   | 0.073 |   0.284 |    2.324 |
     | uart_rx_unit/g1708        | A v -> Y ^  | INVX1     | 0.043 |   0.327 |    2.367 |
     | uart_rx_unit/g1681        | C ^ -> Y v  | NAND3XL   | 0.040 |   0.367 |    2.407 |
     | uart_rx_unit/g1668        | C v -> Y ^  | NAND3XL   | 0.031 |   0.398 |    2.439 |
     | uart_rx_unit/g1662        | B ^ -> Y v  | NAND2XL   | 0.021 |   0.420 |    2.460 |
     | uart_rx_unit/g1660        | B0 v -> Y ^ | OAI21XL   | 0.014 |   0.433 |    2.474 |
     | uart_rx_unit/n_reg_reg[1] | D ^         | DFFRHQX1  | 0.000 |   0.433 |    2.474 |
     +-------------------------------------------------------------------------------+
```



The resultant density was **93.14%**

5 Reflection on Individual and Teamwork (PO(i))

5.1 Individual Contribution of Each Member

The team consisted of four members, each contributing unique skills and expertise to the project:

1. Literature review	2006108,2006109,2006163,2006171
2. Directed testbench, layered verification, functional coverage	2006109,2006163,2006171
3. Synthesis and Area, Cell Optimization	2006109, 2006108
4. Physical Design	2006109,2006163
5. Die Optimization	2006109,2006108, 2006163,2006171
6. Report writing	2006108,2006109,2006163,2006171

5.2 Mode of Team Work

The team worked from the very beginning and became successful. The team had a lot of diversity. The individuals worked in different areas and then all six had cascaded the circuits to build the project.

5.3 Diversity Statement of Team

Our team brought together members with diverse academic backgrounds, technical skills, and problem-solving approaches, which greatly benefited the development of the UART communication interface project. Each member contributed in their area of expertise—some focused on digital design and RTL coding, while others handled simulation, verification, and documentation. This combination of strengths fostered a collaborative environment where different perspectives led to more efficient solutions, enhanced learning, and a deeper understanding of UART design and implementation.

6 Communication to External Stakeholders (PO(j))

6.1 Executive Summary

The UART (Universal Asynchronous Receiver/Transmitter) communication interface project focuses on designing, implementing, and verifying a reliable digital module for serial data transmission. The project covers RTL design, layered verification using SystemVerilog, and synthesis for gate-level implementation. A modular testbench with generator, driver, monitor, and scoreboard ensures functional correctness and thorough coverage of data transactions, including corner and random cases. The synthesis process converts the RTL design into a gate-level netlist while meeting timing, area, and power constraints, producing reports for timing, power, area, and quality-of-results. The collaborative efforts of the team, combining expertise in digital design, simulation, verification, and documentation, led to a robust and fully verified UART design ready for practical deployment in digital systems.

7 Future Work (PO(l))

This project involved the design, implementation, and verification of a UART (Universal Asynchronous Receiver/Transmitter) communication interface. The design was realized in RTL, verified using a layered SystemVerilog testbench, and synthesized to a gate-level netlist with timing, area, and power reports. The UART module supports basic asynchronous serial communication with data transmission and reception verified through functional coverage, ensuring reliability across corner cases. While the current design is functional and verified, several enhancements can be made for broader applicability, higher efficiency, and integration into complex digital systems.

1. Implement variable baud rate support to allow flexible communication speeds for different applications.
2. Add parity bits, error detection, and correction mechanisms to increase reliability in noisy communication environments.
3. Extend the module to full-duplex operation, enabling simultaneous transmission and reception of data.
4. Integrate FIFO buffers to handle burst data efficiently and prevent data loss during high-speed communication.
5. Optimize the design for lower area and power, making it suitable for FPGA or ASIC deployment in embedded systems.
6. Develop a configurable UART that can support different word lengths (5–9 bits) and stop bit options.
7. Combine the UART module with other serial protocols like SPI or I2C to form a versatile communication IP block.
8. Apply formal verification methods alongside simulation to ensure exhaustive correctness of the UART design.

9. Incorporate IoT or remote monitoring features to track communication status, data integrity, and module performance in real-time.

8 References

[1] V. V. S. Raghava and M. Ravi Kumar, “Digital system design of FPGA-based UART protocol using Verilog HDL,” *International Journal of Computational and Experimental Science and Engineering*, 2025.

[2] A. K. Singh and D. Pal, “Universal asynchronous receiver transmitter (UART),” *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 2024.

[3] Kavyashree S., “Design and implementation of UART using Verilog,” *International Journal of Engineering and Computer Science*, vol. 4, no. 5, pp. 11753–11757, May 2015.

[4] T. Chandupatla, R. Karthik, and S. Prasad, “UART IP core design and verification using SystemVerilog,” *International Journal of Engineering and Computer Science*, 2024.

[5] M. Srinath and S. Hiremath, “Verification of universal asynchronous receiver and transmitter (UART) using SystemVerilog,” *International Journal of Engineering Research & Technology (IJERT)*, 2022.

[6] B. Kumari, A. Sharma, and P. Verma, “High-speed low-power UART design using Verilog,” *International Research Journal of Advanced Engineering and Technology*, 2024.

[7] L. Jin, “Design of anti-interference UART receiver,” *Computers and Artificial Intelligence Journal*, 2025.

[8] “Design and implementation of UART serial communication module based on FPGA,” in *Proceedings of the International Conference on Mechatronics, Electronics and Information Engineering (ICMEIE)*, 2015.

[9] J. Axelson, *Serial Port Complete*, 2nd ed. Madison, WI, USA: Lakeview Research, 2007.

[10] P. P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Hoboken, NJ, USA: Wiley, 2008.

[11] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann, 2012.